

On the Adaptive Security of FROST

Elizabeth Crites¹, Jonathan Katz², Chelsea Komlo³, Stefano Tessaro⁴,
and Chenzhi Zhu⁴

¹ Web3 Foundation elizabeth@web3.foundation

² Google, USA jkatz2@gmail.com

³ University of Waterloo, NEAR One, Waterloo, Canada ckomlo@uwaterloo.ca

⁴ Paul G. Allen School of Computer Science & Engineering
University of Washington, Seattle, US
{zhucz20,tessaro}@cs.washington.edu

Abstract. FROST and its variants are state-of-the-art protocols for threshold Schnorr signatures that are used in real-world applications. While *static* security of these protocols has been shown by several works, the security of these protocols under *adaptive* corruptions—where an adversary can choose which parties to corrupt at any time based on information it learns during protocol executions—has remained a notorious open problem that has received renewed attention due to recent standardization efforts for threshold schemes.

We show adaptive security (without erasures) of FROST and several variants under different corruption thresholds and computational assumptions. Let n be the total number of parties, $t + 1$ the signing threshold, and t_c an upper bound on the number of corrupted parties.

1. We prove adaptive security when $t_c = t/2$ in the random-oracle model (ROM) based on the algebraic one-more discrete-logarithm assumption (AOMDL)—the same conditions under which FROST has been proven statically secure.
2. We introduce the *low-dimensional vector representation* (LDVR) problem, parameterized by t_c , t , and n , and prove adaptive security in the algebraic-group model (AGM) and ROM based on the AOMDL assumption and the hardness of the LDVR problem for the corresponding parameters. In some regimes (including some $t_c > t/2$), we show the LDVR problem is unconditionally hard, while in other regimes (in particular, when $t_c = t$), we show that hardness of the LDVR problem is necessary for adaptive security to hold. In fact, we show that hardness of the LDVR problem is necessary for proving adaptive security of a broad class of threshold Schnorr signatures.

Keywords: Threshold Signatures, Schnorr Signatures, Adaptive Security, Random-Oracle Model, Algebraic-Group Model

Table of Contents

1	Introduction	3
1.1	Related Work	4
1.2	Future Work	5
2	Preliminaries	6
2.1	Threshold Signatures	7
2.2	The FROST Protocols	10
3	Adaptive Security of FROST for $t/2$ Corruptions	12
4	Full Adaptive Security of FROST	14
4.1	The Low-Dimensional Vector Representation Problem	14
4.2	Necessity of the LDVR Assumption for Adaptive Security	16
4.3	Proof of Full Adaptive Security	18
4.4	Hardness of LDVR	26
4.5	Cryptanalysis of the LDVR Problem	30
A	Proof of Adaptive Security for $t/2$ Corruptions	35
B	Proof of Lemma 2	41

1 Introduction

A $(t + 1, n)$ -threshold signature scheme allows a signing key to be distributed among n parties such that any $t + 1$ of those parties can jointly generate a signature on a message, while an adversary corrupting up to some specified number $t_c \leq t$ of those parties is unable to do so. Motivated primarily by blockchain applications, several works have considered threshold protocols for ECDSA, Schnorr, and BLS signatures in recent years.

FROST and its variants [KG20, CKM21, BTZ22, BCK⁺22, RRJ⁺22, CGRS23] (hereinafter collectively referred to as FROST) represent state-of-the-art protocols for threshold Schnorr signatures; a version has been codified in an IETF RFC [CKGW22] and is currently used in real-world applications [GKC22]. Security of FROST in a *static* corruption model—where the adversary decides at the outset which parties to corrupt—has been shown to hold in the works cited above. However, proving security of these protocols under *adaptive* corruptions—where the adversary can choose which parties to corrupt at any time based on information it learns from executions of the protocol—has remained a challenging open problem.¹ This problem has become more pressing with the recent NIST call for threshold protocols [BD22, BP25], which highlights adaptive security (for up to $n = 1024$ parties) as a consideration. We remark that while there has been recent work designing other threshold Schnorr protocols with adaptive security, those schemes are not currently deployed, are less efficient than FROST, and have other disadvantages (see Section 1.1 for further discussion); therefore, it is still important to understand the security of FROST itself.

In this work, we show several results regarding the adaptive security of FROST. First, we prove adaptive security at half the maximal corruption threshold (i.e., when $t_c = t/2$) in the random-oracle model (ROM) based on the algebraic one-more discrete-logarithm assumption (AOMDL)—the same conditions under which FROST is known to be statically secure. Our proof methodology here is similar to the one used to analyze Sparkle+ [CKM23]: we invoke the forking lemma and exploit the fact that the total number of corruptions in both the “original” execution and the “rewound” execution is bounded by t . While this result holds only for a sub-optimal corruption threshold, it nevertheless fills an important gap in the literature.²

We also prove adaptive security of FROST (under stronger conditions) for t_c beyond $t/2$. Inspired by recent work [CS] proposing an adaptive strategy

¹ Note that adaptive security implies static security. It is also trivial to show (by simply guessing in advance the parties to be corrupted) that any statically secure scheme is adaptively secure with a multiplicative security loss of $\binom{n}{t_c}$. When n, t_c are large, however, this term is super-polynomial.

² Note that the NIST call for threshold protocols [BD22, BP25] explicitly allows for consideration of sub-optimal corruption thresholds, especially when there are other advantages (e.g., in terms of required assumptions or efficiency).

for attacking a broad class³ of threshold Schnorr protocols (including FROST), we introduce a problem that we call the *low-dimensional vector representation* (LDVR) problem, parameterized by t_c, t , and n . If the LDVR problem is easy for particular values of t_c, t, n , then we show there is an explicit (adaptive) adversary corrupting t_c parties who can violate the security of FROST with threshold $t + 1$; in other words, hardness of the LDVR problem is *necessary* for adaptive security of FROST to possibly hold. Complementing this result, we show that hardness of the LDVR problem for particular values of t_c, t, n is sufficient to prove security of FROST (with signing threshold $t + 1$) for up to t_c adaptive corruptions, under the same conditions as above but now also in the algebraic-group model (AGM) [FKL18]. (We remark that, prior to our work, no pairing-free, two-round threshold signature scheme with adaptive security was known, even assuming the AGM.) Roughly speaking, then, we isolate the LDVR assumption and show that its hardness is tightly connected to the adaptive security of FROST.

Because of the centrality of the LDVR problem to the adaptive security of FROST, we initiate study of the hardness of this problem. For some t_c, t, n —including some values of t_c beyond $t/2$ —we show that the LDVR problem is *unconditionally* hard. In other regimes, (computational) hardness of the LDVR problem must be assumed. We justify the assumption in that case by showing that some natural approaches to solving the problem fail.

1.1 Related Work

Prior works have investigated the adaptive security of threshold protocols for Schnorr [AF04, Mak22, CKM23, BLSW24, BDLR24, KRT24], BLS [BL22, DR23], ECDSA [CCL⁺23, KGG24], RSA [ADN06], and other signature schemes [BLT⁺23, BBC⁺25]. In this work, we focus on threshold Schnorr signatures; all the schemes discussed are in this setting as well.

Abe and Fehr [AF04] give an adaptively secure protocol with maximal corruption threshold (i.e., $t_c = t$) based on the decisional Diffie-Hellman (DDH) assumption (and security of Schnorr signatures). Their scheme has at least three rounds, relies on strong network assumptions (synchronous broadcast), and requires an honest majority (i.e., $t_c < n/2$). FROST tolerates a dishonest majority and works in an asynchronous network, without broadcast or even authenticated channels.

Crites et al. [CKM23] introduce **Sparkle+** and show that it is adaptively secure for $t/2$ corruptions based on the AOMDL assumption in the ROM, and adaptively secure for t corruptions based on the AOMDL assumption in the AGM+ROM.⁴

³ The attack applies to any scheme where the i th party holds a Shamir share $\mathbf{sk}_i$ of the secret key and the “commitment” $g^{\mathbf{sk}_i}$ can be learned by an adversary. See Section 4.2.

⁴ The work cited previously [CS] shows that full adaptive security of **Sparkle+** (among other schemes) cannot be proven without an assumption that implies the hardness of some instances of a search problem P introduced in their work. We similarly show this for the LDVR problem.

Like FROST, Sparkle+ allows for pre-processing before the message or signing set is known, but requires two online signing rounds and authenticated channels.

Zero S. [Mak22] is adaptively secure for t corruptions in the *erasure* model, where honest parties are allowed to securely erase state. It is proven to UC-realize the threshold signature functionality [CGG⁺20] in the global ROM, assuming the hardness of the discrete-logarithm problem. Zero S. is a three-round protocol and makes use of online-extractable zero-knowledge proofs [Fis05], which adds overhead.

HARTS [BLSW24] is a constant-round scheme achieving robustness and adaptive security in the asynchronous setting. However, their security proof only considers the super honest-majority setting (i.e., $t_c < n/3$), and the protocol has high round complexity.

Katsumata et al. [KRT24] prove adaptive security of two schemes, Crackle and Snap, for t corruptions. These schemes do not have identifiable abort, whereas FROST does [RRJ⁺22]. Moreover, the schemes require five and four signing rounds, respectively, as well as authenticated channels.

Crites et al. [CKM25] prove a series of impossibility results for adaptive security of threshold signature schemes with a certain *key-uniqueness* property. In particular, they show that key-unique threshold Schnorr signature schemes cannot be proven adaptively secure beyond $t/2$ corruptions under the (A)OMDL assumption in the programmable ROM using a fully black-box reduction that rewinds the adversary and programs at least one random-oracle query. This is exactly the type of reduction we employ in our adaptive security proof against at most $t/2$ corruptions, and so aligns with their result.

Glacius [BDLR24] is a five-round protocol that is proven adaptively secure for t corruptions based on DDH in the ROM, assuming authenticated channels. Interestingly, the key shares (and commitments thereto) have a different structure than what is required for the aforementioned attack [CS] and impossibility result [CKM25].

Relationship to [CS]. Our work was inspired by the recent work of Crites and Stewart [CS] showing an adaptive strategy for attacking a class of threshold Schnorr protocols including FROST (cf. footnote 3). Their work can be viewed as showing an *efficient* attack given access to an oracle solving a particular computational problem P whose hardness is unclear. The main message of their work is that an efficient algorithm for solving P would give an efficient adaptive attack on FROST, in particular, and thus show an example of a natural protocol that is statically secure but not adaptively secure.

In contrast, our focus is on proving adaptive security of FROST under well-defined assumptions. (We stress that Crites and Stewart are silent about whether assumed hardness of problem P could be used to prove adaptive security of FROST.) To do so, we formalize the LDVR problem that is related to problem P , and show that the LDVR problem is sufficient (along with other assumptions) and necessary to prove adaptive security of FROST. We also analyze the hardness of the LDVR problem, and show that it is unconditionally hard in certain parameter regimes. See Section 4.1 for further discussion.

1.2 Future Work

Hardness of the LDVR problem (for parameters where our unconditional hardness result does not hold) is unclear, and requires further investigation. Note that any $o(\sqrt{p})$ -time algorithm solving the LDVR problem (where p is the order of the underlying group) would be interesting, as it would imply an attack on FROST better than what is currently known using generic discrete-logarithm algorithms. Beyond the practical implications for FROST, such a result would also be interesting insofar as it would show a natural example of a protocol where there is a gap between the known advantages of adaptive and static attackers.

2 Preliminaries

We let $\kappa \in \mathbb{N}$ denote the security parameter. We let $x \leftarrow^* S$ denote sampling a uniform element of a set S and assigning it to x . We use $[n]$ to represent $\{1, \dots, n\}$ and $[i..j]$ to represent $\{i, \dots, j\}$. We represent vectors as $\vec{a} = (a_1, \dots)$.

PPT stands for “probabilistic polynomial time.” Algorithms are randomized unless explicitly noted otherwise. We let $y := A(x; \rho)$ denote running algorithm A on input x and randomness ρ and assigning its output to y . Let $y \leftarrow^* A(x)$ denote $y := A(x; \rho)$ for a uniform ρ . We let GrpGen be a PPT algorithm that takes as input 1^κ and outputs a description $(\mathbb{G}, p, g)$ of a group $\mathbb{G}$ of order a prime $p > 2^\kappa$, and a generator g of $\mathbb{G}$.

Definition 1 (Schnorr signatures [Sch91]). *The Schnorr signature scheme consists of PPT algorithms (Setup, KeyGen, Sign, Verify) defined as follows:*

- **Setup**(1^κ) $\rightarrow$ **par**: On input 1^κ , run $(\mathbb{G}, p, g) \leftarrow \text{GrpGen}(1^\kappa)$ and select a hash function $H : \{0, 1\}^* \rightarrow \mathbb{Z}_p$. Output public parameters **par** := $((\mathbb{G}, p, g), H)$ (which are given implicitly as input to all other algorithms).
- **KeyGen**() $\rightarrow$ (PK, sk): Sample a secret key $\text{sk} \leftarrow^* \mathbb{Z}_p$ and compute the public key as $\text{PK} := g^{\text{sk}}$. Output key pair (PK, sk).
- **Sign**(sk, m) $\rightarrow$ σ : On input a secret key sk and a message m , sample a nonce $r \leftarrow^* \mathbb{Z}_p$. Then, compute a commitment $R := g^r$, challenge $c := H(R, \text{PK}, m)$, and response $z := r + c \cdot \text{sk}$. Output signature $\sigma := (R, z)$.
- **Verify**(PK, m, σ) $\rightarrow$ 0/1: On input a public key PK, a message m , and a purported signature $\sigma = (R, z)$, compute $c := H(R, \text{PK}, m)$ and output 1 (accept) iff the signature verifies as $R \cdot \text{PK}^c = g^z$.

Schnorr signatures are secure under the discrete-logarithm assumption in the random-oracle model [PS00] and the algebraic-group model [FKL18].

Polynomial interpolation. A polynomial $f(x) = a_0 + a_1x + a_2x^2 + \dots + a_tx^t$ of degree t over a field $\mathbb{F}$ can be interpolated from its value on $t + 1$ points. For distinct elements $S = \{x_1, \dots, x_{t+1}\}$ in $\mathbb{F}$, define the Lagrange polynomial as

$$L_i^{(S)}(x) = \prod_{j \in S, j \neq i} \frac{x - x_j}{x_i - x_j}. \quad (1)$$

Given $(x_i, y_i)_{i \in [t+1]}$, we can implicitly evaluate the corresponding polynomial f at any point x as

$$f(x) = \sum_{k \in [t+1]} f(x_k) \cdot L_k^{(S)}(x).$$

Definition 2 (Shamir secret sharing [Sha79]). *The $(t+1, n)$ -Shamir secret sharing scheme over a field $\mathbb{F}$ consists of efficient algorithms (Share, Recover), defined as follows:*

- **Share**($\text{sk}, n, t+1$) $\rightarrow \{(x_1, \text{sk}_1), \dots, (x_n, \text{sk}_n)\}$: *On input a secret sk , number of participants n , and threshold $t+1$, first, define a polynomial $f(x) = \text{sk} + a_1x + a_2x^2 + \dots + a_tx^t$ by sampling $a_1, \dots, a_t \leftarrow^* \mathbb{Z}_p$. Choose distinct elements $x_1, \dots, x_n$ in $\mathbb{F}$. (These could, for example, be participant indices $1, \dots, n$, which is the convention we use.) Then, set each participant's share $\text{sk}_i, i \in [n]$, to be the evaluation of $f(i)$:*

$$\text{sk}_i := \text{sk} + \sum_{j \in [t]} a_j i^j.$$

Output $\{(x_i, \text{sk}_i)\}_{i \in [n]}$.

- **Recover**($t+1, \{(x_i, \text{sk}_i)\}_{i \in \mathcal{S}}$) $\rightarrow \perp / \text{sk}$: *On input threshold $t+1$ and a set of shares $\{(x_i, \text{sk}_i)\}_{i \in \mathcal{S}}$, output $\perp$ if $\mathcal{S} \not\subseteq [n]$ or if $|\mathcal{S}| < t+1$. Otherwise, recover sk as follows:*

$$\text{sk} := \sum_{i \in \mathcal{S}} \lambda_i^{\mathcal{S}} \text{sk}_i$$

where the Lagrange coefficient for the set $\mathcal{S}$ is defined as

$$\lambda_i^{\mathcal{S}} = \prod_{j \in \mathcal{S}, j \neq i} \frac{j}{j-i}.$$

Algebraic one-more discrete-logarithm (AOMDL) assumption. The AOMDL assumption was introduced formally in [NRS21], though it was used implicitly in prior work [BP02, NKDM03, BS07, FPS20]. As in the standard OMDL experiment, the adversary is given as input $(\mathbb{G}, p, g)$ and group elements $(X_1, \dots, X_\ell)$. The adversary can then query an oracle allowing it to learn specific linear combinations of the discrete logarithms of those group elements. The adversary succeeds if it can compute all the discrete logarithms $(x_1, \dots, x_\ell)$ while making fewer than ℓ queries.

Assumption 1 (AOMDL) *Define*

$$\text{Adv}_{\mathcal{A}, \text{GrpGen}}^{\ell\text{-aomdl}}(\kappa) = \Pr[\text{Expt}_{\mathcal{A}, \text{GrpGen}}^{\ell\text{-aomdl}}(\kappa) = 1],$$

where $\text{Expt}_{\mathcal{A}, \text{GrpGen}}^{\ell\text{-aomdl}}(\kappa)$ is defined in Figure 1. The ℓ -AOMDL assumption holds relative to GrpGen if $\text{Adv}_{\mathcal{A}, \text{GrpGen}}^{\ell\text{-aomdl}}(\kappa)$ is negligible for all PPT adversaries $\mathcal{A}$.

MAIN $\text{Expt}_{\mathcal{A}, \text{GrpGen}}^{\ell\text{-aomdl}}(\kappa)$	$\mathcal{O}^{\text{dl}}(\alpha, (\beta_i)_{i=1}^\ell)$
$(\mathbb{G}, p, g) \leftarrow \text{GrpGen}(1^\kappa)$	if $\text{ctr} = \ell - 1$, return $\perp$
$\text{ctr} := 0$	$\text{ctr} := \text{ctr} + 1$
for $i \in [\ell]$ do	$x := \alpha + \sum_{i=0}^t \beta_i \cdot x_i$
$x_i \leftarrow \mathbb{Z}_p$; $X_i := g^{x_i}$	return x
$\vec{x} := (x_1, \dots, x_\ell)$	
$\vec{X} := (X_1, \dots, X_\ell)$	
$\vec{x}' := \mathcal{A}^{\mathcal{O}^{\text{dl}}}((\mathbb{G}, p, g), \vec{X})$	
if $\vec{x}' = \vec{x}$	
return 1	
return 0	

Fig. 1. Experiment for defining the AOMDL assumption.

2.1 Threshold Signatures

We begin with the definition of a threshold signature scheme, adapted to 2-round, “partially non-interactive” schemes (like FROST), where the first round can be executed before the message or signing set are determined. In this work, we abstract out the distribution of key material to the parties and focus on signing only; thus, we represent key generation via a centralized algorithm (that could be run by a dealer or implemented by a secure distributed key-generation protocol).

Definition 3. A threshold signature scheme consists of algorithms *Setup*, *KeyGen*, *Sign₁*, *Sign₂*, *PreAgg*, *Combine*, and *Verify*, with the following syntax:

- $\text{Setup}(1^\kappa) \rightarrow \text{par}$: Takes as input a security parameter and outputs public parameters *par* (given implicitly as input to all other algorithms).
- $\text{KeyGen}(n, t+1) \rightarrow (\text{PK}, \{\text{PK}_i\}_{i \in [n]}, \{\text{sk}_i\}_{i \in [n]})$: Takes as input the number of signers n and the threshold $t+1 \leq n$ and outputs a public key *PK*, a set $\{\text{PK}_i\}_{i \in [n]}$ of parties’ public keys, and a set $\{\text{sk}_i\}_{i \in [n]}$ of secret shares.
- $(\text{Sign}_1, \text{Sign}_2) \rightarrow \{\text{pm}_{i,1}, \text{pm}_{i,2}\}_{i \in \mathcal{S}}$: These represent the two rounds of a signing protocol. We have

$$\begin{aligned}
 (\text{pm}_{i,1}, \text{st}_{i,1}) &:= \text{Sign}_1(i, \text{sk}_i) \\
 \text{pm}_{i,2} &:= \text{Sign}_2(i, \text{st}_{i,1}, \mathcal{S}, m, \mathcal{PM}_1),
 \end{aligned}$$

where $\text{pm}_{i,1}, \text{pm}_{i,2}$ are protocol messages, $\text{st}_{i,1}$ is the state of party i after the first round, $\mathcal{S} \subseteq [n], |\mathcal{S}| \geq t+1$ is a signing set, m is a message to be signed, and $\mathcal{PM}_1 = \{\text{pm}_{i,1}\}_{i \in \mathcal{S}}$.

- $\text{PreAgg}(\mathcal{S}, \mathcal{PM}_1) \rightarrow \mathcal{PM}'_1$: This is an (optional) deterministic algorithm that takes as input the signing set $\mathcal{S}$ and a set of protocol messages $\mathcal{PM}_1$, and

outputs a second set of protocol messages $\mathcal{PM}'_1$. If utilized, this algorithm is performed after Sign_1 and before Sign_2 .

- $\text{Combine}(\mathcal{S}, m, \mathcal{PM}_1, \mathcal{PM}_2) \rightarrow (m, \sigma)$: This is a deterministic algorithm that takes as input the signing set $\mathcal{S}$, the message m , and a set of protocol messages $\mathcal{PM}_1, \mathcal{PM}_2 = \{\text{pm}_{i,2}\}_{i \in \mathcal{S}}$, and outputs a signature σ .
- $\text{Verify}(\text{PK}, m, \sigma) \rightarrow 0/1$: This is a deterministic algorithm that takes as input the public key PK , a message m , and a purported signature σ and outputs 1 (accept) iff the signature verifies.

Although the $\{\text{PK}_i\}_{i \in [n]}$ are not explicitly used in the above definition (nor in this work), they can optionally be used for verification purposes (i.e., identifiable abort). Moreover, many distributed key-generation algorithms reveal $\{\text{PK}_i\}_{i \in [n]}$ during their execution. Finally, we remark that in FROST it is possible for an adversary to derive $\{\text{PK}_i\}_{i \in [n]}$ from executions of the signing protocol.

Correctness. A threshold signature scheme is *correct* if for all κ , all par output by $\text{Setup}(1^\kappa)$, all n and $t < n$, all $\text{PK}, \{(\text{PK}_i, \text{sk}_i)\}_{i \in [n]}$ output by $\text{KeyGen}(n, t + 1)$, all $\mathcal{S} \subseteq [n]$ with $|\mathcal{S}| \geq t + 1$, and all m , we have:

$$\Pr[\sigma \leftarrow \text{TSignHon}(\text{PK}, \{\text{sk}_i\}_{i \in [\mathcal{S}]}, \mathcal{S}, m) : \text{Verify}(\text{PK}, m, \sigma) = 1] = 1,$$

where TSignHon is defined in Fig. 2.

$\text{TSignHon}(\text{PK}, \{\text{sk}_i\}_{i \in [\mathcal{S}]}, \mathcal{S}, m)$ return $\perp$ if $\mathcal{S} \not\subseteq [n] \vee \mathcal{S} < t + 1$ for $i \in \mathcal{S}$ do $(\text{pm}_{i,1}, \text{st}_{i,1}) \leftarrow \text{Sign}_1(i, \text{sk}_i)$ for $i \in \mathcal{S}$ do $\text{pm}_{i,2} \leftarrow \text{Sign}_2(i, \text{st}_{i,1}, \mathcal{S}, m, \mathcal{PM}_1)$ return $\text{Combine}(\mathcal{S}, m, \mathcal{PM}_1, \mathcal{PM}_2)$
--

Fig. 2. Algorithm TSignHon for honestly generating a threshold signature.

Adaptive security. We define adaptive security via the experiment in Figure 3, which we now describe. Key generation is run, and a (stateful) adversary $\mathcal{A}$ is given the public parameters, public key, and parties' public keys. $\mathcal{A}$ can interact with a corruption oracle that provides the adversary with both the secret key and the entire state of any party the adversary chooses to corrupt, subject only to an upper bound t_c on the total number of parties that may be corrupted. We stress here that we do not assume any erasure of parties' states. In addition, the adversary can interact with signing oracles that model the behavior of (honest) parties executing the protocol. Although the signing oracles are defined as if targeting general two-round protocols, the definition actually captures the offline-online (or partially non-interactive) signing pattern, where the first-signing round serves as a preprocessing round that generates nonces before the message to be

MAIN $\text{Expt}_{\mathcal{A}, \text{TS}}^{\text{adp-TS-UF}}(\kappa, n, t, t_c)$	$\mathcal{O}^{\text{Sign}_1}(k)$
$\text{par} \leftarrow \text{Setup}(1^\kappa)$	return $\perp$ if $k \notin \text{hon}$
$\text{Q}_{\text{st}} := \emptyset, \text{Q}_m := \emptyset$	$\text{ctr}_k := \text{ctr}_k + 1$
$(\text{PK}, \{(\text{PK}_i, \text{sk}_i)\}_{i \in [n]}) \leftarrow \text{KeyGen}(n, t + 1)$	$\text{eid} := \text{ctr}_k$
$\text{input} := (\text{par}, \text{PK}, \{\text{PK}_i\}_{i \in [n]})$	$(\text{pm}_{k, \text{eid}, 1}, \text{st}_{k, \text{eid}, 1}) \leftarrow \text{Sign}_1(k, \text{sk}_k)$
for $i \in [n]$ do $\text{ctr}_i := 0$	$\text{Q}_{\text{st}}[k, \text{eid}, 1] := \text{st}_{k, \text{eid}, 1}$
$(m^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}^{\text{Sign}_1, \text{Sign}_2, \text{Corrupt}}}(\text{input})$	return $(\text{eid}, \text{pm}_{k, \text{eid}, 1})$
return 1 if $m^* \notin \text{Q}_m$	$\mathcal{O}^{\text{Sign}_2}(k, \text{eid}, \mathcal{S}, m, \mathcal{PM}_1)$
$\wedge \text{Verify}(\text{PK}, m^*, \sigma^*) = 1$	return $\perp$ if $k \notin \text{hon} \vee \text{Q}_{\text{st}}[k, \text{eid}, 1] = \perp$
return 0	return $\perp$ if $\text{Q}_{\text{st}}[k, \text{eid}, 2] \neq \perp$
$\mathcal{O}^{\text{Corrupt}}(k)$	parse $\text{st}_{k, \text{eid}, 1} := \text{Q}_{\text{st}}[k, \text{eid}, 1]$
return $\perp$ if $(k \notin \text{hon}) \vee (\text{cor} \geq t_c)$	$\text{Q}_m := \text{Q}_m \cup \{m\}$
$\text{cor} := \text{cor} \cup \{k\}$	$\text{pm}_{k, \text{eid}, 2} \leftarrow \text{Sign}_2(k, \text{st}_{k, \text{eid}, 1}, \mathcal{S}, m, \mathcal{PM}_1)$
$\text{hon} := \text{hon} \setminus \{k\}$	$\text{Q}_{\text{st}}[k, \text{eid}, 2] := \text{pm}_{k, \text{eid}, 2}$
$\text{st}_k := \text{Q}_{\text{st}}[k, \cdot, \cdot] \quad // \text{ all state for party } k$	return $\text{pm}_{k, \text{eid}, 2}$
return $(\text{sk}_k, \text{st}_k)$	

Fig. 3. Experiment for defining adaptive unforgeability. The public parameters par are implicitly given as input to all algorithms, and $\mathcal{A}$ is stateful. Here, $t + 1 \leq n$ is the reconstruction threshold and $t_c \leq t$ is the maximum number of signers $\mathcal{A}$ can corrupt.

signed or the set of involved signers is known. Note that we do not assume private or authenticated channels; thus, $\mathcal{A}$ can see all messages honest parties send (even before any parties are corrupted) and send messages of its choice to honest parties in the second round of the protocol. Furthermore, our model allows for a *concurrent* adversary who may open multiple signing sessions simultaneously.

At the end of the adversary's execution, it succeeds if it outputs a forgery, i.e., a valid message/signature pair (relative to PK) for a message that was not previously queried to any party. (Our definition corresponds to TS-UF-0 from the hierarchy of definitions in [BTZ22, BCK⁺22].)

Definition 4 (Adaptive Security). *For an adversary $\mathcal{A}$ and threshold signature scheme TS, define*

$$\text{Adv}_{\mathcal{A}, \text{TS}}^{\text{adp-TS-UF}}(\kappa, n, t, t_c) = \Pr[\text{Expt}_{\mathcal{A}, \text{TS}}^{\text{adp-TS-UF}}(\kappa, n, t, t_c) = 1].$$

TS is (n, t, t_c) -adaptively secure if for all PPT adversaries $\mathcal{A}$, the advantage $\text{Adv}_{\mathcal{A}, \text{TS}}^{\text{adp-TS-UF}}(\kappa, n, t, t_c)$ is negligible. (We stress that n, t, t_c may depend on κ .) TS has full adaptive security if it is (n, t, t) -adaptively secure for any $t < n$.

2.2 The FROST Protocols

FROST1, FROST2, and FROST3 are all 2-round threshold Schnorr signature schemes where the first round can be run during a pre-processing phase before the message or signing set are known. FROST1 [KG20] contains the core ideas of the scheme. FROST2 [CKM21, BTZ22, BCK⁺22] provides a computational optimization of FROST1. (It was later observed [CG23] that the use of multi-scalar multiplication allows the computational efficiency of FROST1 to be comparable to FROST2. We include FROST2 for cases where such optimizations are not possible.) FROST3 [RRJ⁺22, CGRS23] improves the communication complexity of FROST2.

We describe the signing protocol of FROST1, and then highlight the differences in FROST2 and FROST3. (As discussed previously, we leave key generation out of scope.) An overview of all the schemes is given in Figure 4.

Parameter generation (Setup). On input 1^κ , the parameter-generation algorithm runs $(\mathbb{G}, p, g) \leftarrow \text{GrpGen}(1^\kappa)$ and selects hash functions $H_{\text{non}}, H_{\text{sig}} : \{0, 1\}^* \rightarrow \mathbb{Z}_p$. It outputs public parameters $\text{par} \leftarrow ((\mathbb{G}, p, g), H_{\text{non}}, H_{\text{sig}})$.

Key generation (KeyGen). On input the total number of signers n and the threshold $t + 1$, the key-generation algorithm chooses a secret key $\text{sk} \leftarrow \mathbb{Z}_p$ and sets the public key to $\text{PK} := g^{\text{sk}}$. It then performs a Shamir secret sharing of sk , computing $\{(i, \text{sk}_i)\}_{i \in [n]} \leftarrow \text{Share}(\text{sk}, n, t + 1)$. The corresponding public key for each participant is $\text{PK}_i := g^{\text{sk}_i}$. It outputs $(\text{PK}, \{(\text{PK}_i, \text{sk}_i)\}_{i \in [n]})$.

Signing round 1 (Sign₁). On input a participant identifier k , party k samples nonces $r_k, s_k \leftarrow \mathbb{Z}_p$ and sets $R_k := g^{r_k}$ and $S_k := g^{s_k}$. It outputs (R_k, S_k) .

Signing round 2 (Sign₂). On input a participant identifier k , signing set $\mathcal{S}$, message m , and set of nonce commitments $\mathcal{PM}_1 = \{(i, R_i, S_i)\}_{i \in \mathcal{S}}$, party k first checks that their nonce pair (i.e., $(k, R_k, S_k) \in \{(i, R_i, S_i)\}_{i \in \mathcal{S}}$) is what is recorded in their state from round 1. If so, it computes aggregate nonce commitment $\tilde{R} := \text{CompR}(\text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$, $a := H_{\text{non}}(k, \text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$, $c := H_{\text{sig}}(\tilde{R}, \text{PK}, m)$, and partial signature $z_k := r_k + s_k a + c \lambda_k^{\mathcal{S}} \text{sk}_k$ (where $\lambda_k^{\mathcal{S}}$ is the Lagrange coefficient for party k in $\mathcal{S}$). It outputs z_k .

Computing aggregate nonce (CompR). On input $\text{PK}, \mathcal{S}, m$, and $\mathcal{PM}_1$, where $\mathcal{PM}_1 = \{(i, R_i, S_i)\}_{i \in \mathcal{S}}$, the nonce aggregation algorithm computes $a_i := H_{\text{non}}(i, \text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$ and aggregate nonce $\tilde{R} := \prod_{i \in \mathcal{S}} R_i \cdot S_i^{a_i}$.

Combining partial signatures (Combine). On input $\mathcal{S}, m, \mathcal{PM}_1$, and partial signatures $\mathcal{PM}_2 = \{z_i\}_{i \in \mathcal{S}}$, the combiner (who may be one of the signers or an external party) computes the aggregate nonce $\tilde{R} := \text{CompR}(\text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$ and $z := \sum_{i \in \mathcal{S}} z_i$ and sets $\sigma := (\tilde{R}, z)$. It outputs σ .

Verification (Verify). On input PK, m, σ , the signature is verified exactly as in the Schnorr signature scheme.

FROST2 uses a single scalar a (instead of a scalar a_i for each party). FROST3 has an additional algorithm **PreAgg**, which allows for pre-aggregation of nonces, reducing the communication overhead.

We write $\text{FROST1}[\text{GrpGen}]$, etc., when we wish to emphasize that the scheme uses a particular group-generation algorithm **GrpGen**.

Static security of FROST. Security of FROST1 for static corruptions was first claimed by Komlo and Goldberg [KG20]. However, they only gave a security proof for a less-efficient variant of FROST1, arguing heuristically that the result extends to the original protocol. Bellare et al. [BCK⁺22] later gave a rigorous proof of security for FROST1 and FROST2 based on the one-more discrete-logarithm assumption. Ruffing et al. [RRJ⁺22] prove the static security of FROST3 via a reduction to the static security of FROST2; Chu et al. [CGRS23] prove the static security of FROST3 with respect to a concrete distributed key-generation protocol.

3 Adaptive Security of FROST for $t/2$ Corruptions

We prove adaptive security of all three variants of FROST for a maximum corruption threshold of $t_c = t/2$ based on the AOMDL assumption in the random-oracle model.

Theorem 1. *Let GrpGen be a group generator. For $t < n$ and $t_c = \lfloor t/2 \rfloor$, any $\text{FROST} \in \{\text{FROST1}[\text{GrpGen}], \text{FROST2}[\text{GrpGen}], \text{FROST3}[\text{GrpGen}]\}$, and any PPT adversary $\mathcal{A}$ making at most q_s queries to $\mathcal{O}^{\text{Sign}_1}$ and at most q_h random-oracle queries, there exists a PPT adversary $\mathcal{B}$ such that*

$$\text{Adv}_{\mathcal{A}, \text{FROST}}^{\text{adp-TS-UF}}(\kappa, n, t, t_c) \leq \sqrt{q \cdot \left(\text{Adv}_{\mathcal{B}, \text{GrpGen}}^{(2q_s + t + 1)\text{-aomdl}}(\kappa) + \frac{3q^2}{2^\kappa} \right)} + \frac{q}{2^\kappa}, \quad (2)$$

where $q = n(q_s + q_h + 1)$ for FROST1 and $q = q_s + q_h + 1$ for FROST2 and FROST3. The running time of $\mathcal{B}$ is roughly twice that of $\mathcal{A}$.

The full proof can be found in Appendix A. The proofs for the three variants of FROST are largely the same; hence, we provide a unified proof and highlight the points where the proofs diverge.

Proof Outline. Let $\mathcal{A}$ be a PPT adversary against the $t/2$ adaptive unforgeability of FROST (Fig. 3). We construct a PPT reduction $\mathcal{B}$ against the $(2q_s + t + 1)$ -aomdl assumption (Fig. 1). We show that if $\mathcal{A}$ produces a forgery with non-negligible probability, then $\mathcal{B}$ breaks the $(2q_s + t + 1)$ -aomdl assumption with non-negligible probability.

The reduction $\mathcal{B}$ takes as input $(2q_s + t + 1)$ -aomdl challenges $(X_0, \hat{X}_1, \dots, \hat{X}_t, X_1, X'_1, \dots, X_{q_s}, X'_{q_s})$ and aims to output x_i such that $X_i = g^{x_i}$ for all i without querying its discrete-logarithm solution oracle more than $2q_s + t$ times. It runs a forking algorithm on a simulator algorithm $\mathcal{C}$ that simulates the unforgeability game to $\mathcal{A}$ as follows.

$\mathcal{C}$ is given as input the AOMDL challenges and $q (= q_s + q_h + 1)$ random values. For all $i \in [n]$, $\mathcal{C}$ sets each public key share as $\text{PK}_i := X_0 \hat{X}_1^i \dots \hat{X}_t^i$. The threshold public key is $\text{PK} := X_0$. $\mathcal{C}$ then returns $(\text{PK}, \{\text{PK}_i\}_{i \in [n]})$ to $\mathcal{A}$. $\mathcal{C}$ simulates random-oracle queries using the q random values. $\mathcal{C}$ simulates each query to $\mathcal{O}^{\text{Sign}_1}$ by returning a pair (X_i, X'_i) , and simulates each query to $\mathcal{O}^{\text{Sign}_2}$

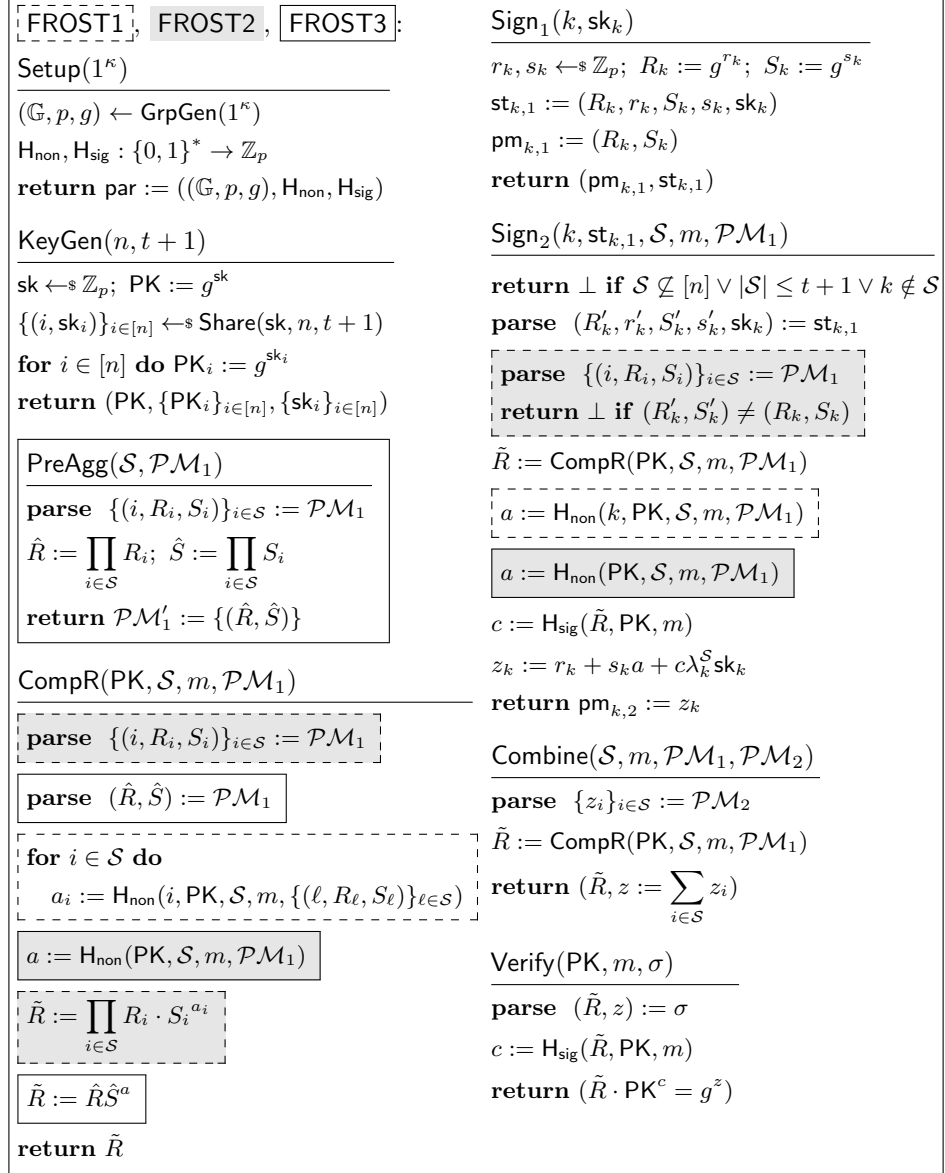


Fig. 4. The FROST1, FROST2, and FROST3 threshold signature schemes, where FROST1 contains only the dashed boxes, FROST2 contains only the highlighted boxes, and FROST3 contains only the solid boxes. The public parameters **par** and public key **PK** are implicitly given as input to all algorithms. **CompR** is an algorithm that computes the aggregate nonce $\tilde{R}$ from a second-round input.

by making a single query for the DL of $g^{z_k} = X_i X_i'^a \text{PK}_k^{c\lambda_k^S}$ to obtain z_k for party k . $\mathcal{C}$'s DL solution queries are forwarded to $\mathcal{B}$'s DL solution oracle.

$\mathcal{C}$ is forked in order to extract $\text{sk} = x_0$ from $\mathcal{A}$'s two forgeries. Assume w.l.o.g. that $\mathcal{A}$ corrupts t parties over the two iterations. ($\mathcal{A}$ can corrupt up to $t/2$ parties in each iteration, and $\mathcal{C}$ can corrupt the remaining itself.) Then $\mathcal{C}$, and therefore $\mathcal{B}$, has made t DL solution queries on $g^{\text{sk}_k} = X_0 \hat{X}_1^k \cdots \hat{X}_t^k$ to obtain sk_k . $\mathcal{B}$ now knows $t + 1$ points on the polynomial $f(Z) = x_0 + \hat{x}_1 Z + \cdots + \hat{x}_t Z^t$ defining key generation. Thus, $\mathcal{B}$ can solve for $\hat{x}_1, \dots, \hat{x}_t$. For the remaining $(x_1, x'_1, \dots, x_{q_s}, x'_{q_s})$, $\mathcal{B}$ first considers nonce pairs (X_i, X'_i) that remain honest at the end of the game. (1) If no $\mathcal{O}^{\text{Sign}_2}$ query has been made on (X_i, X'_i) in either iteration of $\mathcal{A}$, $\mathcal{B}$ queries each DL directly. (2) If one $\mathcal{O}^{\text{Sign}_2}$ query has been made, $\mathcal{B}$ has already made one DL solution query, for z_k , and makes one more for X_i . $\mathcal{B}$ can then solve for the DL of X'_i itself from $z_k = x_i + x'_i a + c\lambda_k^S \text{sk}_k$, given that it knows sk_k or can compute it as $f(k)$. If two $\mathcal{O}^{\text{Sign}_2}$ queries have been made, one in each iteration, there are two cases. (3) If the query occurs before the fork, then it is the same in both iterations and this is the same as case (2). (4) If the query occurs after the fork, then we have:

$$x'_i = \frac{z_{k'_j} - z_{k_j} + c_j \gamma_{k_j} \text{sk}_{k_j} - c_{j'} \gamma_{k_{j'}} \text{sk}_{k'_{j'}}}{a_{j'} - a_j}$$

where $a_{j'} \neq a_j$ due to forking, and $\mathcal{B}$ then solves for x_i from z_k . $\mathcal{B}$ then considers nonce pairs (X_i, X'_i) that are corrupted by the end of the game. When a party is corrupted, $\mathcal{B}$ queries its DL solution oracle on PK_k to get sk_k . If no $\mathcal{O}^{\text{Sign}_2}$ query has been made on (X_i, X'_i) , $\mathcal{B}$ queries for each DL directly. This is the same as case (1); the cases (2), (3), and (4) are similar. Thus, $\mathcal{B}$ has made the same queries for these cases at the time of corruption. Overall, $\mathcal{B}$ has made exactly two DL queries for each nonce pair (X_i, X'_i) . Thus, $\mathcal{B}$ has solved for $(x_0, \hat{x}_1, \dots, \hat{x}_t, x_1, x'_1, \dots, x_{q_s}, x'_{q_s})$ using only $2q_s + t$ DL solution queries to win the $(2q_s + t + 1)$ -aomdl game.

4 Full Adaptive Security of FROST

In this section we introduce the low-dimensional vector representation (LDVR) problem and the corresponding assumption that the problem is hard. In Section 4.2 we then show that the LDVR assumption is necessary for adaptive security of a broad class of threshold Schnorr schemes (including FROST), in the sense that an efficient algorithm solving the LDVR problem implies an explicit adaptive attack on those schemes. In Section 4.3, we prove full adaptive security of FROST under the AOMDL and LDVR assumptions in the AGM and ROM. In Section 4.4 we study the hardness of the LDVR problem itself, showing that, for some parameters, the problem is unconditionally hard, and in other cases providing evidence that the problem is computationally hard.

MAIN $\text{Expt}_{\mathcal{A}}^{(t_c, t, n)\text{-ldvr}}(\kappa)$	$\mathcal{O}(\vec{\alpha})$
ctr := 0	// $\vec{\alpha} \in \mathbb{Z}_p^{n+1}$
$(p, st) \leftarrow \mathcal{A}(\kappa)$	ctr := ctr + 1
// $2^\kappa < p < 2^{\kappa+1}$, p prime	$\vec{\alpha}_{\text{ctr}} := \vec{\alpha}$
for $j \in [0..n]$ do	$c_{\text{ctr}} \leftarrow \mathbb{Z}_p$
$\vec{v}_j := (1, j, \dots, j^t) \in \mathbb{Z}_p^{t+1}$	return c_{ctr}
$(\text{CS}, i) \leftarrow \mathcal{A}^{\mathcal{O}}(st)$	
// $\text{CS} \subseteq [n]$, $ \text{CS} \leq t_c$, $i \in [\text{ctr}]$	
$\vec{w} := c_i \vec{v}_0 + \sum_{j=0}^n \vec{\alpha}_i[j] \cdot \vec{v}_j$	
if $\vec{w} \in \text{span}(\{\vec{v}_i\}_{i \in \text{CS}})$	
return 1	
return 0	

Fig. 5. The LDVR experiment with parameters $t_c \leq t < n$.

4.1 The Low-Dimensional Vector Representation Problem

The *low-dimensional vector-representation (LDVR) problem* is parameterized by t_c, t, n with $t_c \leq t < n$. (Looking ahead to the application to threshold schemes, n will correspond to the total number of parties, $t+1$ to the signing threshold, and t_c to the corruption bound.) Roughly speaking, an adversary $\mathcal{A}$ is given vectors $\vec{v}_0, \dots, \vec{v}_n \in \mathbb{Z}_p^{t+1}$ and is then asked to find $\vec{\alpha} \in \mathbb{Z}_p^{n+1}$ such that the vector

$$\vec{w} := \mathcal{O}(\vec{\alpha}) \cdot \vec{v}_0 + \sum_{i=0}^n \alpha[i] \cdot \vec{v}_i$$

lies in a t_c -dimensional subspace of $\mathbb{Z}_p^{t+1}$, where $\mathcal{O} : \mathbb{Z}_p^{n+1} \rightarrow \mathbb{Z}_p$ can be viewed as a random oracle. More specifically, in the formal experiment defining the problem (cf. Figure 5) the $\vec{v}_0, \dots, \vec{v}_n$ are “Vandermonde vectors” $\vec{v}_i = (1, i, \dots, i^t)$; moreover, $\mathcal{A}$ must specify a set $\text{CS} \subseteq [n]$ of size t_c defining the t_c -dimensional subspace $\text{span}(\{\vec{v}_i\}_{i \in \text{CS}})$ in which $\vec{w}$ must lie. We also allow $\mathcal{O}$ to be queried multiple times with the same input; each such query results in a fresh random output.

We denote the advantage of $\mathcal{A}$ in solving the (t_c, t, n) -LDVR problem as

$$\text{Adv}_{\mathcal{A}}^{(t_c, t, n)\text{-ldvr}}(\kappa) = \Pr \left[\text{Expt}_{\mathcal{A}}^{(t_c, t, n)\text{-ldvr}}(\kappa) = 1 \right].$$

We formulate in particular the following assumption.

Definition 5 (LDVR Assumption). *We say the (t_c, t, n) -LDVR assumption holds (where t_c, t, n may be functions of κ) if $\text{Adv}_{\mathcal{A}}^{(t_c, t, n)\text{-ldvr}}(\kappa)$ is negligible for any PPT algorithm $\mathcal{A}$.*

We analyze the hardness of this problem in Section 4.4. There, we show that the assumption holds unconditionally for several interesting values of t_c, t, n . When $t_c = t$, it seems reasonable to conjecture that solving the problem requires time $\Omega(\sqrt{p})$ (as hinted by our preliminary cryptanalysis in Section 4.5), which roughly matches the conjectured hardness of the discrete-logarithm problem in a group of order p on a well-chosen elliptic curve.

Relationship to prior work [CS]. Prior work [CS] shows an attack against a large class of Schnorr threshold signature schemes (the same class we discuss in the next section) given access to an oracle $\mathcal{O}_P$ that takes an arbitrary vector $\vec{w} \in \mathbb{Z}_p^{t+1}$ as input and returns a subset $\text{CS} \subseteq [n]$ of size t_c such that $\vec{w} \in \text{span}(\{\vec{u}_i\}_{i \in \text{CS}})$, if such a subset exists. The problem solved by the oracle $\mathcal{O}_P$ is referred to as problem P . (We refer to $\vec{w}$ for which such a subset CS exists as *good*.) Clearly, the LDVR assumption cannot hold relative to $\mathcal{O}_P$ if any of the $\vec{w}$ -vectors defined by a query to $\mathcal{O}$ during the adversary’s execution in the LDVR experiment are good. For this reason, the ideas from [CS] give an efficient algorithm for the LDVR problem given access to $\mathcal{O}_P$, as their analysis implies that when the $\vec{\alpha}$ queried to $\mathcal{O}$ are uniform, the resulting vector $\vec{w}$ is good with high probability (depending on t_c, t , and n). Thus, with the same probability, $\mathcal{O}_P$ can be used to find a suitable set CS such that $\vec{w} \in \text{span}(\{\vec{v}_i\}_{i \in \text{CS}})$, and thus solve the LDVR problem. In the case $t_c = t$, they show that this happens with probability $1/2$ if (roughly) $\binom{n}{t} > p$; their argument extends to $t_c < t$, albeit with a sharp drop in probability.

Conversely, if the resulting vector $\vec{w}$ is *good* with negligible probability, the LDVR problem is unconditionally hard. In Section 4.4, we show an explicit upper bound on this probability, which shows for certain choices of (t_c, t, n) , the LDVR problem is unconditionally hard. However, it is unclear whether problem P is hard or not in those scenarios.⁵

Summarizing: prior work [CS] shows an oracle $\mathcal{O}_P$ relative to which there is an attack on both a certain class of Schnorr threshold schemes and the LDVR problem for certain choices of (t_c, t, n) . That is, if P is easy, LDVR is easy for the same parameter regimes where the attack from [CS] applies. We do not know whether our unconditional analysis is tight; thus, there is a range of parameter choices where it is possible that LDVR is hard, but P is easy. And of course, it is always possible that LDVR is easy, but P is hard in the worst case. In Section 4.2, we show that an oracle solving the LDVR problem also suffices to implement the attack on the same class of Schnorr threshold schemes.

4.2 Necessity of the LDVR Assumption for Adaptive Security

Here, we show that if the (t_c, t, n) -LDVR problem is easy, there is an explicit adaptive attack on a broad class of threshold Schnorr schemes for the same parameters. The attack does not require the adversary to request any signatures; it also does not require the adversary to learn anything about the parties’ internal

⁵ Here, we refer to the worst-case hardness of P .

states other than their secret keys, so applies even if secure erasure is assumed. This shows that the LDVR assumption is necessary if there is to be any hope of proving such schemes adaptively secure.

Specifically, we consider protocols with the following properties:

- The secret-key shares of the parties are Shamir shares of the master secret key sk , with the i th party holding share sk_i .
- “Commitments” $\{\text{PK}_i = g^{\text{sk}_i}\}_{i \in [n]}$ can be computed by the adversary. Note that for the purposes of the attack it does not matter whether these commitments are required by the protocol itself, are available to the adversary because of the distributed key-generation (DKG) protocol used, or computable from other information available to the adversary.

The above conditions apply to a wide range of schemes [Lin24, RRJ⁺22, TZ23, CKM23, KG24] in addition to FROST and its variants. For the case of FROST in particular, note that (1) although commitments to the parties’ key shares are not required to run the signing protocol, such commitments are assumed to be publicly available in the IETF RFC [CKGW22] and are necessary for identifiable abort; in any event (2) commitments to the parties’ key shares can be computed from honest executions of the FROST signing protocol. Moreover, (3) we are not aware of any dlog-based DKG protocol that does not reveal these commitments.

Fix t_c, t, n for which the (t_c, t, n) -LDVR is easy, and let $\mathcal{A}'$ be an efficient algorithm solving it. Consider a Schnorr threshold scheme of the type above with n parties and signing threshold $t + 1$. We show an adversary $\mathcal{A}$ (similar to the prior one [CS]) who corrupts at most t_c parties and forges a signature. At the outset, $\mathcal{A}$ is given $\text{PK} = g^{\text{sk}}$ and $\{\text{PK}_i = g^{\text{sk}_i}\}_{i \in [n]}$, where $\text{sk}_i = a(i)$ for a polynomial $a(X) = a_0 + a_1X + \dots + a_tX^t$ with $a_0 = \text{sk}$. Then $\mathcal{A}$ runs $\mathcal{A}'$ (we assume here that $\mathcal{A}'$ uses the same prime p used by the scheme). When $\mathcal{A}'$ issues its i th query to $\mathcal{O}$ with input $\vec{\alpha}_i$, adversary $\mathcal{A}$ computes

$$R_i := \text{PK}^{\vec{\alpha}_i[0]} \cdot \prod_{j=1}^n \text{PK}_i^{\vec{\alpha}_i[j]}$$

followed by $c_i := \text{H}_{\text{sig}}(R_i, \text{PK}, m_i)$ where m_i is distinct from $\{m_j\}_{j < i}$. $\mathcal{A}$ returns c_i to $\mathcal{A}'$. When $\mathcal{A}'$ outputs (CS, i^*) , adversary $\mathcal{A}$ obtains $\{\text{sk}_i\}_{i \in \text{CS}}$ by corrupting all signers $i \in \text{CS}$, and (via linear algebra) computes $(\beta_j)_{j \in \text{CS}}$ such that

$$\vec{w}_{i^*} \stackrel{\text{def}}{=} c_{i^*} \cdot \vec{v}_0 + \sum_{j=0}^n \vec{\alpha}_{i^*}[j] \cdot \vec{v}_j = \sum_{j \in \text{CS}} \beta_j \cdot \vec{v}_j.$$

Finally, $\mathcal{A}$ outputs $(m_{i^*}, \sigma = (R_{i^*}, z_{i^*}))$ where $z_{i^*} = \sum_{j \in \text{CS}} \beta_j \cdot \text{sk}_j$.

If H_{sig} is modeled as a random oracle, the view of $\mathcal{A}'$ is the same as its view in $\text{Expt}_{\mathcal{A}'}^{(t_c, t, n)\text{-ldvr}}(\kappa)$. Moreover, $\mathcal{A}$ succeeds when $\mathcal{A}'$ does. To see this, let $\langle \cdot, \cdot \rangle$ denote the dot product and note that if we let $\vec{a} = [a_0, a_1, \dots, a_t]$ be the vector

of coefficients of the sharing polynomial a , then $\mathbf{sk}_i = \langle \vec{a}, \vec{v}_i \rangle$. We have

$$\begin{aligned} z_{i^*} &= \sum_{j \in \text{CS}} \beta_j \cdot \langle \vec{a}, \vec{v}_j \rangle = \left\langle \vec{a}, \sum_{j \in \text{CS}} \beta_j \cdot \vec{v}_j \right\rangle \\ &= \langle \vec{a}, \vec{w}_{i^*} \rangle \\ &= (c_{i^*} + \vec{\alpha}_{i^*}[0]) \cdot \mathbf{sk} + \sum_{j=1}^n \vec{\alpha}_{i^*}[j] \cdot \mathbf{sk}_j, \end{aligned}$$

which is exactly the discrete logarithm of $R_{i^*} \cdot \text{PK}^{c_{i^*}}$. Thus, (R_{i^*}, z_{i^*}) is a valid signature on m_{i^*} .

4.3 Proof of Full Adaptive Security

We prove that full adaptive security of all three variants of FROST is implied by the hardness of the LDVR problem and the AOMDL assumption in the algebraic-group model and the random-oracle model.

Theorem 2. *Let GrpGen be a group generator. For all $t_c \leq t < n$, any $\text{FROST} \in \{\text{FROST1}[\text{GrpGen}], \text{FROST2}[\text{GrpGen}], \text{FROST3}[\text{GrpGen}]\}$, and any algebraic PPT adversary $\mathcal{A}$ making at most q_s queries to Sign_1 and at most q_h random-oracle queries, there is a PPT adversary $\mathcal{B}$ and a PPT adversary $\mathcal{C}$, who makes at most q queries to its oracle, such that*

$$\text{Adv}_{\mathcal{A}, \text{FROST}}^{\text{adp-TS-UF}}(\kappa, n, t, t_c) \leq \text{Adv}_{\mathcal{B}, \text{GrpGen}}^{(2q_s + t + 1)\text{-aomdl}}(\kappa) + \text{Adv}_{\mathcal{C}}^{(t_c, t, n)\text{-ldvr}}(\kappa) + \frac{2(nq)^2}{2\kappa}, \quad (3)$$

where $q = q_s + q_h + 1$. The running times of $\mathcal{B}$ and $\mathcal{C}$ are roughly the same as that of $\mathcal{A}$.

The security proofs for the three FROST variants are largely the same; hence, we provide a unified proof and highlight the places where the proofs diverge.

Proof Outline. Suppose $\mathcal{A}$ outputs a valid forgery $(m^*, \sigma^* = (R^*, z^*))$ satisfying $g^{z^*} = R^* \cdot \text{PK}^{c^*}$ with $c^* = \text{H}_{\text{sig}}(R^*, \text{PK}, m^*)$. We show that given a successful adversary, we can either define an efficient reduction to AOMDL, or an efficient reduction to the LDVR problem, using the following steps. The rough idea of the AOMDL reduction can be described as follows. The reduction first sets the public key PK and public key shares $\{\text{PK}_k\}_{k \in [n]}$ using $(t+1)$ AOMDL challenges $X_0, \dots, X_t$, where $\text{PK} := X_0$ and $\text{PK}_k = X_0 \prod_{i \in [t]} X_i^{k_i}$. Then, the reduction runs the adversary $\mathcal{A}$ with the public key and public key shares by simulating the signing and corruption oracles on its own. For $\mathcal{O}^{\text{Sign}_1}(k)$, the reduction returns (R_k, S_k) , which it sets as two new AOMDL challenges. For $\mathcal{O}^{\text{Sign}_2}$ and $\mathcal{O}^{\text{Corrupt}}$, the reduction simulates the responses by querying its own $\mathcal{O}^{\text{dl}}$ oracle. After $\mathcal{A}$ returns a valid forgery as described above, the reduction attempts to extract a *new* linear relation among the challenges from the forgery, which helps it to win

the AOMDL game. Here, “new” means that the relation does not lie in the linear span of the $\mathcal{O}^{\text{dl}}$ oracle queries made by the reduction. More precisely, there are the following three cases.

1. We first show one possible strategy by which the AOMDL reduction can win. Given a forgery as described above, since $\mathcal{A}$ is algebraic, it must provide a representation of R^* with respect to $g, \text{PK}, \{\text{PK}_k\}_{k \in [n]}$, and all the nonce commitments $(R_{i,k}, S_{i,k})$ output by honest signers $k \in \text{hon}$ during executions of the protocol, where $(R_{i,k}, S_{i,k})$ is the i -th pair sent by the k -th signer. The AOMDL reduction can replace each term in the representation of R^* involving $(R_{i,k}, S_{i,k})$ (denoted by $R_{i,k}^{\gamma_{i,k}} S_{i,k}^{\gamma'_{i,k}}$) with a term involving g and PK_k alone. This is possible if (1) there is a Sign_2 query corresponding to this nonce commitment pair for which the oracle outputs z_k satisfying $g^{z_k} = R_{i,k} S_{i,k}^a \cdot \text{PK}_k^{c\lambda_k}$, and (2) $\gamma'_{i,k} = a \cdot \gamma_{i,k}$. If one of these conditions does not hold for some (i, k) , then the equation $g^{z^*} = R^* \cdot \text{PK}^{c^*}$ gives a new linear relation among $R_{i,k}, S_{i,k}$, and other group elements. From this, the AOMDL reduction can solve for a non-trivial discrete-logarithm solution without querying its own $\mathcal{O}^{\text{dl}}$ oracle.
2. If both conditions (1) and (2) hold for all $(R_{i,k}, S_{i,k})$, then $R_{i,k}^{\gamma_{i,k}} S_{i,k}^{\gamma'_{i,k}}$ can be replaced with $(g^{z_k} \text{PK}_k^{-c\lambda_k})^{\gamma_{i,k}}$, and we show a second strategy by which an AOMDL reduction can win. The AOMDL reduction again defines R^* with the representation described in the step above, resulting in a representation of R^* with respect to $g, \text{PK}, \{\text{PK}_k\}_{k \in [n]}$ only, i.e., we have $R^* = g^\delta \text{PK}^{\beta_0} \prod_{k=1}^n \text{PK}_k^{\beta_k}$ for known $\delta, \beta_0, \{\beta_k\}_{k=1}^n$. Since $g^{z^*} = R^* \cdot \text{PK}^{c^*}$, we have

$$\text{PK}^{\beta_0+c^*} \prod_{k=1}^n \text{PK}_k^{\beta_k} = g^{-\delta+z^*}. \quad (4)$$

Denote $\vec{v}_k := (1, k, k^2, \dots, k^t)$ for $k \in [0..n]$ and $\vec{v}^* = c^* \vec{v}_0 + \sum_{k \in [0..n]} \beta_k \vec{v}_k$. Since $\text{PK}_k = \prod_{j=0}^t X_j^{v_{k,j}}$ and $\text{PK} = \prod_{j=0}^t X_j^{v_{0,j}}$ (recalled that $X_0, \dots, X_t$ are $(t+1)$ AOMDL challenges), by Equation (4),

$$\prod_{j=0}^t X_j^{v_j^*} = g^{-\delta+z^*}. \quad (5)$$

Then, if $\vec{v}^* \notin \text{span}(\{\vec{v}_j\}_{j \in \text{cor}})$, Equation (5) gives a new linear relation among $X_0, \dots, X_t$. Therefore, the AOMDL reduction can again solve for a non-trivial discrete-logarithm solution without querying its own $\mathcal{O}^{\text{dl}}$ oracle.

3. If the check in Step 2 fails, i.e., $\vec{v}^* \in \text{span}(\{\vec{v}_j\}_{j \in \text{cor}})$, then $\mathcal{A}$ can compute $\log_g \left(\text{PK}^{\beta_0+c^*} \prod_{k=1}^n \text{PK}_k^{\beta_k} \right)$ using $\{\text{sk}_j\}_{j \in \text{cor}}$, and thus the AOMDL reduction cannot hope to obtain a new linear relation from the above equation. However, in this case, we can define a second reduction that solves the LDVR problem. In particular, $(\beta_0, \dots, \beta_n)$ can be seen as an oracle query in the (t_c, t, n) -LDVR game with response c^* , and since $|\text{cor}| \leq t_c$ and $c^* \vec{v}_0 + \sum_{k \in [0..n]} \beta_k \vec{v}_k \in$

$\text{span}(\{\vec{v}_j\}_{j \in \text{cor}})$, this gives a valid solution for the LDVR problem. The formal reduction is more complicated. In particular, a key challenge is that the $\{\beta_k\}_{k \in [0..n]}$ need to be extracted *at the time* $\mathcal{A}$ makes the random-oracle query to $H_{\text{sig}}(R^*, \text{PK}, m^*)$. We refer to the proof of Lemma 1 for details.

Proof (of Theorem 2). Let $\mathcal{A}$ be an algebraic PPT adversary in the adaptive unforgeability experiment $\text{Exp}_{\mathcal{A}, \text{FROST}}^{\text{adp-TS-UF}}(\kappa, n, t, t_c)$ that makes up to q_h queries to H_{non} and H_{sig} , up to q_s queries to Sign_1 , and up to t_c queries to Corrupt . We assume, without loss of generality, that $\mathcal{A}$ queries H_{sig} on its forgery (R^*, PK, m^*) , and that it always queries $H_{\text{non}}(\text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$ (resp., $H_{\text{non}}(k, \text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$ for FROST1) before querying $\text{Sign}_1(k, \mathcal{S}, m, \mathcal{PM}_1)$ for some $k \in \mathcal{S} \cap \text{hon}$. We now define a PPT algorithm $\mathcal{B}$ for the AOMDL experiment.

$\mathcal{B}$ is initialized by the AOMDL challenger with the group description $(\mathbb{G}, g, p)$ and a set of $2q_s + t + 1$ AOMDL challenges $(X_0, \hat{X}_1, \dots, \hat{X}_t, X_1, X'_1, \dots, X_{q_s}, X'_{q_s})$. $\mathcal{B}$ has access to a discrete-logarithm solution oracle $\mathcal{O}^{\text{dl}}$, and its goal is to output discrete logarithms $(x_0, \hat{x}_1, \dots, \hat{x}_t, x_1, x'_1, \dots, x_{q_s}, x'_{q_s})$ without querying $\mathcal{O}^{\text{dl}}$ more than $2q_s + t$ times. $\mathcal{B}$ begins by initializing the following values:

1. Tables $\mathbf{Q}_{\text{non}} := \emptyset$, $\mathbf{Q}_{\text{sig}} := \emptyset$ to store random-oracle queries to H_{non} , H_{sig} and corresponding simulated responses;
2. Table $\mathbf{Q}_m := \emptyset$, to store the set of messages queried by $\mathcal{A}$ to Sign_2 ;
3. Table $\mathbf{Q}_{\text{st}} := \emptyset$ of participants' states for all signing sessions initiated by $\mathcal{A}$;
4. Counter $\text{ctr} := 0$ to iterate through pairs of AOMDL challenges;
5. Counter $\text{ctr}_k := 0$ for $k \in [n]$ to count the number of Sign_1 queries for party k .

To simulate key generation, $\mathcal{B}$ uses the AOMDL challenges $(X_0, \hat{X}_1, \dots, \hat{X}_t)$ to define a polynomial $f(Z) = x_0 + \hat{x}_1 Z + \dots + \hat{x}_t Z^t$ “in the exponent” by deriving each party’s public key as:

$$\text{PK}_i = X_0 \hat{X}_1^i \dots \hat{X}_t^{i^t}, \quad (6)$$

where implicitly $\text{sk}_i = f(i)$. The threshold public key is $\text{PK} := X_0$ with corresponding secret key $\text{sk} = x_0 = f(0)$. $\mathcal{B}$ then runs $\mathcal{A}((\mathbb{G}, g, p), \text{PK}, \{\text{PK}_i\}_{i \in [n]})$ with the random oracles and signing oracles simulated as follows.

Random oracles: These are simulated in the natural way, choosing a uniform output for each fresh query made by $\mathcal{A}$. Queries/responses to H_{non} (resp., H_{sig}) are stored in $\mathbf{Q}_{\text{non}}$ (resp., $\mathbf{Q}_{\text{sig}}$).

$\mathcal{O}^{\text{Sign}_1}$ **queries:** When $\mathcal{A}$ queries Sign_1 for party $k \in \text{hon}$, algorithm $\mathcal{B}$ does the following:

1. Increment $\text{ctr}_k := \text{ctr}_k + 1$ and set $\text{eid} := \text{ctr}_k$.
2. Increment $\text{ctr} := \text{ctr} + 1$ and set $(R_k, S_k) := (X_{\text{ctr}}, X'_{\text{ctr}})$.
3. Set $\text{st}_{k, \text{eid}, 1} := (R_k, \perp, S_k, \perp, \perp)$.
4. Update $\mathbf{Q}_{\text{st}}[k, \text{eid}, 1] := \text{st}_{k, \text{eid}, 1}$.

Finally, $\mathcal{B}$ returns (R_k, S_k) to $\mathcal{A}$.

$\mathcal{O}^{\text{Sign}_2}$ **queries:** When $\mathcal{A}$ queries $\mathcal{O}^{\text{Sign}_2}(k, \text{eid}, \mathcal{S}, m, \mathcal{PM}_1)$ for $k \in \text{hon}$, algorithm $\mathcal{B}$ first performs the checks in the $\mathcal{O}^{\text{Sign}_2}$ oracle (Fig. 3) and the Sign_2 algorithm (Fig. 4). If they pass, then in particular $\text{st}_{k, \text{eid}, 1} = (R_k, \perp, S_k, \perp, \perp)$. Then $\mathcal{B}$ does the following:

1. Set $\mathbf{Q}_m := \mathbf{Q}_m \cup \{m\}$.
2. Run the Sign_2 algorithm (Fig. 4) to obtain a and c . Note that $\mathcal{B}$ can internally query $\mathbf{H}_{\text{non}}$ and $\mathbf{H}_{\text{sig}}$ as needed.
3. Set $Z_k := R_k S_k^a \cdot \text{PK}_k^{c\lambda_k}$, i.e., $Z_k = X_{i'}(X_{i'}^a)^a X_0^{c\lambda_k} \hat{X}_1^{c\lambda_k k} \dots \hat{X}_t^{c\lambda_k k^t}$ for some $i' \in [q_s]$. $\mathcal{B}$ then queries $\mathcal{O}^{\text{dl}}$ on Z_k with $\alpha = 0$ and

$$(\beta_i)_{i=1}^{2q_s+t+1} = (c\lambda_k, c\lambda_k k, \dots, c\lambda_k k^t, 0, \dots, 0, 1, a, 0, \dots, 0)$$

(where the 1 is in position $t + 2i'$), and receives z_k .

4. Set $\text{st}_{k,\text{eid},2} := z_k$ and return z_k to $\mathcal{A}$.

Corruptions: When $\mathcal{A}$ corrupts party k , $\mathcal{B}$ sets $\text{cor} := \text{cor} \cup \{k\}$ and $\text{hon} := \text{hon} \setminus \{k\}$. Then $\mathcal{B}$ queries $\mathcal{O}^{\text{dl}}$ on PK_k with $\alpha = 0$ and (cf. Eq. (22))

$$(\beta_i)_{i=1}^{2q_s+t+1} = (1, k, \dots, k^t, 0, \dots, 0)$$

and receives sk_k . Then $\mathcal{B}$ does the following:

1. If no entry $\mathbf{Q}_{\text{st}}[k, \star, 1]$ exists, then $\mathcal{B}$ returns sk_k to $\mathcal{A}$. (This corresponds to the case where $\mathcal{A}$ has not previously queried party k to Sign_1 .)
2. Otherwise
 - (a) For each entry $\mathbf{Q}_{\text{st}}[k, \text{eid}, 1] \neq \perp$ with $\mathbf{Q}_{\text{st}}[k, \text{eid}, 2] = \perp$, (This corresponds to round-1 messages that were never used in round 2.) algorithm $\mathcal{B}$ queries $\mathcal{O}^{\text{dl}}$ on R_k (which is equal to $X_{i'}$ for some $i' \in [q_s]$) with $\alpha = 0$ and $(\beta_i)_{i=1}^{2q_s+t+1} = (0, \dots, 0, 1, 0, \dots, 0)$, where 1 is in position $t + 2i'$; it receives r_k in response. $\mathcal{B}$ similarly queries $\mathcal{O}^{\text{dl}}$ on S_k (with $\alpha = 0$ and $(\beta_i)_{i=1}^{2q_s+t+1} = (0, \dots, 0, 1, 0, \dots, 0)$, where 1 is in position $t + 1 + 2i'$), and receives s_k . Then $\mathcal{B}$ sets $\mathbf{Q}_{\text{st}}[k, \text{eid}, 1] = (R_k, r_k, S_k, s_k, \text{sk}_k)$.
 - (b) For each entry $\mathbf{Q}_{\text{st}}[k, \text{eid}, 2] \neq \perp$, (This corresponds to Round 1 queries that made it to Round 2.) algorithm $\mathcal{B}$ queries $\mathcal{O}^{\text{dl}}$ on S_k (which is equal to $X_{i'+1}'$ for some $i' \in [q_s]$) with $\alpha = 0$ and $(\beta_i)_{i=1}^{2q_s+t+1} = (0, \dots, 0, 1, 0, \dots, 0)$, where 1 is in position $t + 1 + 2i'$; it receives s_k in response. Note that $\mathcal{B}$ must have already made an $\mathcal{O}^{\text{dl}}$ query to obtain z_k , so it can compute $r_k = z_k - s_k a - c\lambda_k \text{sk}_k$. Then $\mathcal{B}$ sets $\mathbf{Q}_{\text{st}}[k, \text{eid}, 1] = (R_k, r_k, S_k, s_k, \text{sk}_k)$.

Finally, $\mathcal{B}$ returns sk_k and all state $\mathbf{Q}_{\text{st}}[k, \cdot, \cdot]$ for party k to $\mathcal{A}$.

Output: When $\mathcal{A}$ terminates with output $(m^*, \sigma^* = (R^*, z^*))$, then $\mathcal{B}$ aborts if $\mathcal{A}$ does not succeed. $\mathcal{B}$ finds $\mathcal{A}$'s corresponding query to $\mathbf{H}_{\text{sig}}$ to obtain $c^* := \mathbf{H}_{\text{sig}}(R^*, \text{PK}, m^*)$. When $\mathcal{A}$ made this query, it must have provided a representation

$$R^* = g^\delta \cdot \text{PK}^\beta \cdot \prod_{k=1}^n \text{PK}_k^{\beta_k} \prod_{i=1}^{q_s} R_{i,1}^{\gamma_{i,1}} S_{i,1}^{\gamma'_{i,1}} \dots R_{i,n}^{\gamma_{i,n}} S_{i,n}^{\gamma'_{i,n}}, \quad (7)$$

based on all group elements it had seen so far of the form, where $(R_{i,k}, S_{i,k})$ denotes the output of the i th query $\text{Sign}(k)$. (Note that the above includes some group elements that $\mathcal{A}$ may not have known at the time it made the

H_{sig} query in question, in which case the corresponding $\gamma_{i,k}, \gamma'_{i,k}$ will be 0.) Let hon (resp., cor) be the set of honest (resp., corrupted) signers at the end of $\mathcal{A}$'s execution. For $i \in [q_s]$ and $k \in \text{hon}$, if $\mathcal{A}$ made the corresponding Sign_2 query (i.e. $\text{Q}_{\text{st}}[k, i, 2] \neq \perp$), then let $(k, \text{eid}, \mathcal{S}, m, \mathcal{PM}_1)$ be that query. Note that, because $\mathcal{B}$ simulated Sign_2 , it knows $(a_{i,k}, z_{i,k}, c_{i,k}, \lambda_{i,k})$ such that

$$g^{z_{i,k}} = R_{i,k} S_{i,k}^{a_{i,k}} \text{PK}_k^{c_{i,k} \cdot \lambda_{i,k}}, \quad (8)$$

where $z_{i,k} = \text{Q}_{\text{st}}[k, i, 2]$, $a_{i,k} = H_{\text{non}}(\text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$, $c_{i,k} = H_{\text{sig}}(\tilde{R}, \text{PK}, m)$ with $\tilde{R} := \text{CompR}(\text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$, and $\lambda_{i,k} = \lambda_k^{\mathcal{S}}$. (For FROST1, $a_{i,k} = H_{\text{non}}(k, \text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$.) Call (i, k) *bad* if $(\gamma_{i,k}, \gamma'_{i,k}) \neq (0, 0)$ and either $\text{Q}_{\text{st}}[k, i, 2] = \perp$ or $\gamma'_{i,k} \neq \gamma_{i,k} a_{i,k}$, and let BadRS be the event that some $(i, k) \in [q_s] \times \text{hon}$ is bad.⁶ (Intuitively, if (i, k) is bad then $\mathcal{B}$ cannot compute a representation of $R_{i,k}^{\gamma_{i,k}} S_{i,k}^{\gamma'_{i,k}}$ with respect to g and PK_k .) We show below that when BadRS occurs $\mathcal{B}$ can succeed in the AOMDL experiment.

If BadRS does not occur, then for every (i, k) either $R_{i,k}^{\gamma_{i,k}} S_{i,k}^{\gamma'_{i,k}} = 1$ or, using Equation (8),

$$R_{i,k}^{\gamma_{i,k}} S_{i,k}^{\gamma'_{i,k}} = R_{i,k}^{\gamma_{i,k}} S_{i,k}^{\gamma_{i,k} a_{i,k}} = g^{z_{i,k}} = \left(g^{z_{i,k}} \text{PK}_k^{-c_{i,k} \lambda_{i,k}} \right)^{\gamma_{i,k}}. \quad (9)$$

Since (R^*, z^*) is a valid signature,

$$g^{z^*} \text{PK}^{-c^*} = g^{\delta} \text{PK}^{\beta} \prod_{k=1}^n \text{PK}_k^{\beta_k} \prod_{i=1}^{q_s} R_{i,1}^{\gamma_{i,1}} S_{i,1}^{\gamma'_{i,1}} \cdots R_{i,n}^{\gamma_{i,n}} S_{i,n}^{\gamma'_{i,n}}. \quad (10)$$

Thus, $\mathcal{B}$ can replace $R_{i,k}^{\gamma_{i,k}} S_{i,k}^{\gamma'_{i,k}}$ with $\left(g^{z_{i,k}} \text{PK}_k^{-c_{i,k} \lambda_{i,k}} \right)^{\gamma_{i,k}}$ in Equation (10) for all $i \in [q_s]$ and $k \in \text{hon}$ with $(\gamma_{i,k}, \gamma'_{i,k}) \neq (0, 0)$ to obtain

$$\begin{aligned} g^{z^*} \text{PK}^{-c^*} &= g^{\delta} \text{PK}^{\beta} \prod_{k=1}^n \text{PK}_k^{\beta_k} \prod_{i=1}^{q_s} R_{i,1}^{\gamma_{i,1}} S_{i,1}^{\gamma'_{i,1}} \cdots R_{i,n}^{\gamma_{i,n}} S_{i,n}^{\gamma'_{i,n}} \\ &= g^{\delta} \text{PK}^{\beta} \prod_{k \in \text{hon}} \text{PK}_k^{\beta_k} \prod_{j \in \text{cor}} \text{PK}_j^{\beta_j} \prod_{i=1}^{q_s} \left(\prod_{k \in \text{hon}} R_{i,k}^{\gamma_{i,k}} S_{i,k}^{\gamma_{i,k} a_{i,k}} \prod_{j \in \text{cor}} R_{i,j}^{\gamma_{i,j}} S_{i,j}^{\gamma'_{i,j}} \right) \\ &= g^{\delta} \text{PK}^{\beta} \prod_{k \in \text{hon}} \text{PK}_k^{\beta_k} \prod_{j \in \text{cor}} \text{PK}_j^{\beta_j} \prod_{i=1}^{q_s} \\ &\quad \cdot \left(\prod_{k \in \text{hon}} g^{z_{i,k} \gamma_{i,k}} \text{PK}_k^{-c_{i,k} \lambda_{i,k} \gamma_{i,k}} \prod_{j \in \text{cor}} R_{i,j}^{\gamma_{i,j}} S_{i,j}^{\gamma'_{i,j}} \right) \\ &= g^{\delta + \sum_{i=1}^{q_s} (\sum_{k \in \text{hon}} z_{i,k} \gamma_{i,k} + \sum_{j \in \text{cor}} (\gamma_{i,j} r_{i,j} + \gamma'_{i,j} s_{i,j}))} \end{aligned}$$

⁶ Note that BadRS is referred to as a bad event because it represents a failure case for the reduction to the LDVR problem, as we will see later.

$$\cdot \text{PK}^\beta \prod_{j \in \text{cor}} \text{PK}_j^{\beta_j} \prod_{k \in \text{hon}} \text{PK}_k^{\beta_k - \sum_{i \in [q_s], \gamma_{i,k} \neq 0} c_{i,k} \lambda_{i,k} \gamma_{i,k}}. \quad (11)$$

(Note that it is enough to require $\gamma_{i,k} \neq 0$ since if $\gamma_{i,k} = 0$ then—because (i, k) is good—we must have $\gamma'_{i,k} = a_{i,k} \gamma_{i,k} = 0$.) Let $\vec{v}_k := (1, k, \dots, k^t) \in \mathbb{Z}_p^{t+1}$ for $k \in [0..n]$ and define

$$\vec{v}^* := (c^* + \beta) \vec{v}_0 + \sum_{k \in \text{cor}} \beta_k \cdot \vec{v}_k + \sum_{k \in \text{hon}} \left(\beta_k - \sum_{i \in [q_s], \gamma_{i,k} \neq 0} c_{i,k} \lambda_{i,k} \gamma_{i,k} \right) \cdot \vec{v}_k. \quad (12)$$

Since we can write $\text{PK} = X_0^{v_{0,0}} \cdot \prod_{j \in [t]} \hat{X}_j^{v_{0,j}}$ and $\text{PK}_k = X_0^{v_{k,0}} \cdot \prod_{j \in [t]} \hat{X}_j^{v_{k,j}}$ for all $k \in [n]$, Equation (11) implies

$$g^{\gamma^*} = X_0^{v_0^*} \cdot \prod_{j \in [t]} \hat{X}_j^{v_j^*}, \quad (13)$$

where $\gamma^* := z^* - \delta - \sum_{i=1}^{q_s} (\sum_{k \in \text{hon}} z_{i,k} \gamma_{i,k} - \sum_{j \in \text{cor}} (\gamma_{i,j} r_{i,j} + \gamma'_{i,j} s_{i,j}))$ is a scalar known by $\mathcal{B}$. Let **Badv** be the event that **BadRS** does not occur and $\vec{v}^* \notin \text{span}(\{\vec{v}_k\}_{k \in \text{cor}})$. We show below that if **Badv** occurs then $\mathcal{B}$ can succeed in the AOMDL experiment.

Completing the description of $\mathcal{B}$, we show next how $\mathcal{B}$ can succeed in the AOMDL experiment when $\mathcal{A}$ succeeds and either **BadRS** or **Badv** occurs. Namely, we show that $\mathcal{B}$ can compute the discrete logarithms of every group element it was given at the outset of the experiment, while making $t + 2q_s$ queries to $\mathcal{O}^{\text{dl}}$.

BadRS occurs. If **BadRS** occurs then there exist indices $(\hat{i}, \hat{k}) \in [q_s] \times \text{hon}$ such that $(\gamma_{\hat{i}, \hat{k}}, \gamma'_{\hat{i}, \hat{k}}) \neq (0, 0)$ and either $\text{Q}_{\text{st}}[\hat{k}, \hat{i}, 2] = \perp$ or $\gamma'_{\hat{i}, \hat{k}} \neq \gamma_{\hat{i}, \hat{k}} a_{\hat{i}, \hat{k}}$. Let $i' \in [q_s]$ be such that $(R_{\hat{i}, \hat{k}}, S_{\hat{i}, \hat{k}}) = (X_{i'}, X'_{i'})$.

First of all, $\mathcal{B}$ can compute $(x_0, \hat{x}_1, \dots, \hat{x}_t)$ using $\{\text{sk}_k\}_{k \in \text{cor}}$ and an additional $t + 1 - |\text{cor}|$ queries to $\mathcal{O}^{\text{dl}}$. This accounts for a total of $t + 1$ queries (since $\mathcal{B}$ already used $|\text{cor}|$ queries to learn $\{\text{sk}_k\}_{k \in \text{cor}}$).

Then, for each (X_i, X'_i) that corresponds to nonce commitments of an corrupted signer, $\mathcal{B}$ knows (x_i, x'_i) from its simulation of the **Corrupt** oracle which involved two queries to $\mathcal{O}^{\text{dl}}$. For each $(X_i, X'_i) \neq (X_{i'}, X'_{i'})$ that corresponds to nonce commitments of an honest signer, $\mathcal{B}$ can compute (x_i, x'_i) using either one $\mathcal{O}^{\text{dl}}$ query if it already made a $\mathcal{O}^{\text{dl}}$ query for (X_i, X'_i) as part of simulating a second-round signing query, or using two $\mathcal{O}^{\text{dl}}$ queries otherwise. (This is done exactly as when $\mathcal{B}$ simulates the **Corrupt** oracle.) Therefore, $\mathcal{B}$ makes two $\mathcal{O}^{\text{dl}}$ queries for each $(X_i, X'_i) \neq (X_{i'}, X'_{i'})$.

Finally, we show that $\mathcal{B}$ needs only one $\mathcal{O}^{\text{dl}}$ query to compute $(x_{i'}, x'_{i'})$ corresponding to $(X_{i'}, X'_{i'})$. Assume $\gamma_{\hat{i}, \hat{k}} \neq 0$. (The case of $\gamma'_{\hat{i}, \hat{k}} \neq 0$ is analogous.) If $\mathcal{B}$ has not yet made any $\mathcal{O}^{\text{dl}}$ query involving $(X_{i'}, X'_{i'})$, then $\mathcal{B}$ queries $\mathcal{O}^{\text{dl}}$ on $X'_{i'}$ to obtain $x'_{i'}$, and then derives $x_{i'}$ from Equation (10) since $\gamma_{\hat{i}, \hat{k}} \neq 0$ and

$\mathcal{B}$ knows the discrete logarithms of all group elements in that equation except for $R_{\hat{i},\hat{k}}$. Otherwise, $\mathcal{B}$ knows $g^{z_{\hat{i},\hat{k}} - c_{\hat{i},\hat{k}} \lambda_{\hat{i},\hat{k}} \text{sk}_{\hat{k}}} = X_{i'}(X'_{i'})^{a_{\hat{i},\hat{k}}}$ (cf. Equation (9)). Since $\gamma'_{\hat{i},\hat{k}} \neq \gamma_{\hat{i},\hat{k}} a_{\hat{i},\hat{k}}$, we see that $\mathcal{B}$ can derive $(x_{i'}, x'_{i'})$ using Equation (10), because $\mathcal{B}$ knows the discrete logarithms of all group elements in that equation except for $(R_{\hat{i},\hat{k}}, S_{\hat{i},\hat{k}})$.

Badv occurs. In this case, $\mathcal{B}$ learns a linear relation among $g, X_0, \hat{X}_1, \dots, \hat{X}_t$ (cf. Equation (13)) that is linearly independent of all of the $\mathcal{O}^{\text{dl}}$ queries it made to simulate **Corrupt** queries. $\mathcal{B}$ can thus compute $(x_0, \hat{x}_1, \dots, \hat{x}_t)$ by making an additional $t - |\text{cor}|$ queries to $\mathcal{O}^{\text{dl}}$. Then, as in the previous case, $\mathcal{B}$ can compute (x_i, x'_i) corresponding to each (X_i, X'_i) using a total of two queries to $\mathcal{O}^{\text{dl}}$ for each such pair.

Since $\mathcal{B}$ perfectly simulates $\text{Expt}_{\mathcal{A}, \text{TS}}^{\text{adp-TS-UF}}$, we have

$$\text{Adv}_{\mathcal{A}, \text{TS}}^{\text{adp-TS-UF}}(\kappa, n, t, t_c) \leq \text{Adv}_{\mathcal{B}}^{(2q_s + t + 1)\text{-aomdl}}(\kappa) + \Pr[\text{Succ} \wedge \neg(\text{BadRS} \vee \text{Badv})],$$

where **Succ** is the event that $\mathcal{A}$ succeeds. Lemma 1 completes the proof. $\square$

Lemma 1. *For $\mathcal{A}$ described as in Theorem 2, there is a PPT adversary $\mathcal{C}$ for the (t_c, t, n) -LDVR problem making at most q queries to its oracle and running in time roughly the same as $\mathcal{A}$ such that*

$$\text{Adv}_{\mathcal{C}}^{(t_c, t, n)\text{-ldvr}}(\kappa) \geq \Pr[\text{Succ} \wedge \neg(\text{BadRS} \vee \text{Badv})] - \frac{2(nq)^2}{2^\kappa}. \quad (14)$$

Proof. We assume, without loss of generality, that $\mathcal{A}$ makes a query to H_{sig} on its forgery $(\tilde{R}^*, \text{PK}, m^*)$ and that it always queries $\text{H}_{\text{non}}(\text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$ (resp. $\text{H}_{\text{non}}(k, \text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$ for FROST1) before querying $\mathcal{O}^{\text{sign}_1}(k, \mathcal{S}, m, \mathcal{PM}_1)$ for some $k \in \mathcal{S}$. This adds $q_s + 1$ additional random-oracle queries. Then, let $q = q_h + q_s + 1$.

Construction of $\mathcal{C}$. To start with, $\mathcal{C}$ samples $(\mathbb{G}, p, g) \leftarrow \text{GrpGen}(\kappa)$ and sends p back to the game (t_c, t, n) -LDVR. Then, $\mathcal{C}$ runs $\mathcal{A}$ by simulating the game $\text{Expt}_{\mathcal{A}, \text{FROST}}^{\text{adp-TS-UF}}(\kappa, n, t, t_c)$ faithfully except that $\mathcal{C}$ initializes tables $\text{Flag}_{\text{non}}, \text{Q}_{\text{non}}, \text{Q}_{\text{sig}}$ to empty and simulates queries to the random oracles H_{non} and H_{sig} as follows.

H_{non} : When $\mathcal{A}$ queries H_{non} on T , for FROST1, $\mathcal{C}$ parses $T = (k, \text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$, and for FROST2 and FROST3, $\mathcal{C}$ parses $T = (\text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$. (Here w.l.o.g. we assume T can be parsed correctly and it is for PK , since otherwise the query on a malformed T would not help the adversary to win the game.) Then, $\mathcal{C}$ checks if $\text{Q}_{\text{non}}[T] = \perp$. If not, $\mathcal{C}$ returns $\text{Q}_{\text{non}}[T]$.

Otherwise, for FROST1, $\mathcal{C}$ does the following:

1. For all $i \in \mathcal{S}$, $\mathcal{C}$ samples $a_i \leftarrow \mathbb{Z}_p$ and stores $\text{Q}_{\text{non}}[(i, \text{PK}, \mathcal{S}, m, \mathcal{PM}_1)] = a_i$. If $\text{Flag}_{\text{non}}[a_i] \neq \perp$, $\mathcal{C}$ aborts, and we refer to this bad event as **BadNon**. Otherwise, $\mathcal{C}$ sets $\text{Flag}_{\text{non}}[a_i] := 1$.
2. Then, $\mathcal{C}$ computes $\tilde{R} := \text{CompR}(\text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$.
3. Finally, $\mathcal{C}$ returns a_k .

For FROST2 and FROST3, $\mathcal{C}$ does the following:

1. $\mathcal{C}$ samples $a \leftarrow \mathbb{Z}_p$ and stores $\mathbf{Q}_{\text{non}}[T] := a$.
2. If $\text{Flag}_{\text{non}}[a] \neq \perp$, $\mathcal{C}$ aborts, and we refer to this bad event as **BadNon**. Otherwise, $\mathcal{C}$ sets $\text{Flag}_{\text{non}}[a] := 1$.
3. $\mathcal{C}$ computes $\tilde{R} := \text{CompR}(\text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$.
4. Finally, $\mathcal{C}$ returns a .

$\mathbf{H}_{\text{sig}}$: When $\mathcal{A}$ queries $\mathbf{H}_{\text{sig}}$ on $(\tilde{R}, \text{PK}, m)$, if $\mathbf{Q}_{\text{sig}}[(\text{PK}, m, \tilde{R})] \neq \perp$, $\mathcal{C}$ returns $\mathbf{Q}_{\text{sig}}[(\text{PK}, m, \tilde{R})]$. (Similar to $\mathbf{H}_{\text{non}}$, here we only consider queries for PK, since otherwise the query would not help the adversary to win the game.) Otherwise, $\mathcal{C}$ queries $\mathcal{O}^H$ to obtain a challenge c on a vector $\vec{\alpha}$, sets $\mathbf{Q}_{\text{sig}}[(\text{PK}, m, \tilde{R})] := c$, and returns c , where $\vec{\alpha}$ is computed as follows.

1. $\mathcal{C}$ learns a representation of $\tilde{R}$ from $\mathcal{A}$ based on all group elements $\mathcal{A}$ had seen so far:

$$\tilde{R} = g^\delta \text{PK}^{\beta_0} \prod_{k=1}^n \text{PK}_k^{\beta_k} \prod_{i=1}^{q_s} R_{i,1}^{\gamma_{i,1}} S_{i,1}^{\gamma'_{i,1}} \cdots R_{i,n}^{\gamma_{i,n}} S_{i,n}^{\gamma'_{i,n}}, \quad (15)$$

where $(R_{i,k}, S_{i,k})$ denotes the output of the i -th query to $\mathcal{O}^{\text{Sign}}(k)$.

2. $\mathcal{C}$ sets $\alpha_k := \beta_k$ for $k \in [0..n]$.
3. For each $i \in [q_s]$ and $k \in \text{hon}$ with $\gamma_{i,k} \neq 0$, $\mathcal{C}$ updates α_k if the following two conditions hold:
 - (a) There exists a $\mathbf{H}_{\text{non}}$ query T such that $\gamma'_{i,k} = \mathbf{Q}_{\text{non}}[T] \cdot \gamma_{i,k}$. Since **BadNon** does not occur (otherwise, the game aborts), such T is unique. For FROST1, $\mathcal{C}$ parses $T = (k, \text{PK}, \mathcal{S}, \hat{m}, \mathcal{PM}_1)$.

For FROST2 and FROST3, $\mathcal{C}$ parses $T = (\text{PK}, \mathcal{S}, \hat{m}, \mathcal{PM}_1)$.

- (b) $(m, \tilde{R}) \neq (\hat{m}, \hat{\tilde{R}})$, where $\hat{\tilde{R}} := \text{CompR}(\text{PK}, \mathcal{S}, \hat{m}, \mathcal{PM}_1)$.

If the above conditions hold, $\mathcal{C}$ sets $\alpha_k := \alpha_k - \gamma_{i,k} \lambda_{i,\mathcal{S}} \mathbf{Q}_{\text{sig}}[\text{PK}, \hat{m}, \hat{\tilde{R}}]$.

4. For each $i \in [q_s]$ and $k \in \text{hon}$ with $\gamma_{i,k} \neq 0$, $\mathcal{C}$ sets $\text{Flag}_{\text{non}}[\gamma'_{i,k}/\gamma_{i,k}] := 1$.

Output. At the end of the game, after receiving $(m^*, \sigma^* = (R^*, z^*))$ from $\mathcal{A}$, $\mathcal{C}$ aborts if $\mathcal{A}$ does not win the unforgeability game or $(\text{Badv} \vee \text{BadRS})$ occurs. Otherwise, $\mathcal{C}$ looks up $\mathcal{A}$'s query to $\mathbf{H}_{\text{sig}}$ on (R^*, PK, m^*) and its corresponding query to $\mathcal{O}^H$ on the vector $\vec{\alpha}^*$ such that $\mathbf{H}_{\text{sig}}(R^*, \text{PK}, m^*)$ is set to $\mathcal{O}^H(\vec{\alpha}^*)$. Suppose it is the i -th $\mathcal{O}^H$ query, $\mathcal{C}$ returns (cor, i) .

Analysis of $\mathcal{C}$. We first show $\mathcal{C}$ wins the game LDVR if $\mathcal{C}$ does not abort. Since $\mathcal{A}$ corrupts at most t_c parties, we know $|\text{cor}| \leq t$. Let $c^* := \mathcal{O}^H(\vec{\alpha}^*)$ and $\vec{w}^* := c^* \vec{v}_0 + \sum_{k \in [n]} \alpha_{i,k} \vec{v}_k$. Since $(\text{Badv} \vee \text{BadRS})$ does not occur, $\vec{v}^* \in \text{span}(\{\vec{v}_k\}_{k \in \text{cor}})$. We just need to show that $\vec{w}^* - \vec{v}^* \in \text{span}(\{\vec{v}_k\}_{k \in \text{cor}})$.

When $\mathcal{A}$ made the $\mathbf{H}_{\text{sig}}$ query (R^*, PK, m^*) , it provided a presentation as shown in Equation (7). Denote hon (resp. cor) as the set of honest (resp. corrupted)

parties after $\mathcal{A}$ returned, and denote hon' (resp. cor') as the set of honest (resp. corrupted) parties when $\mathcal{A}$ made the H_{sig} query. From the execution of $\mathcal{C}$,

$$\vec{w}^* := (c^* + \beta)\vec{v}_0 + \sum_{k \in \text{hon}'} \left(\beta_k - \sum_{i \in [q_s], \gamma_{i,k} \neq 0} b_{i,k} \right) \cdot \vec{v}_k, \quad (16)$$

where $b_{i,k} := \gamma_{i,k} \lambda_{i,\mathcal{S}} \text{Q}_{\text{sig}}[\text{PK}, \hat{m}, \hat{R}]$ denotes the decrement applied to α_k in the last line of Step 3 on behalf of (i, k) and $b_{i,k} := 0$ if one of the in Step 3 does not hold. By Equation (12),

$$\begin{aligned} \vec{w}^* - \vec{v}^* &:= - \sum_{k \in \text{cor}} \beta_k \cdot \vec{v}_k - \sum_{k \in \text{hon}' \setminus \text{hon}} \left(\sum_{i=1}^{q_s} b_{i,k} \right) \cdot \vec{v}_k \\ &\quad + \sum_{k \in \text{hon}} \left(\sum_{i \in [q_s], \gamma_{i,k} \neq 0} (c_{i,k} \lambda_{i,k} \gamma_{i,k} - b_{i,k}) \right) \cdot \vec{v}_k. \end{aligned} \quad (17)$$

Since $\text{hon}' \setminus \text{hon} \in \text{cor}$, we have $\sum_{k \in \text{cor}} \beta_k \cdot \vec{v}_k + \sum_{k \in \text{hon}' \setminus \text{hon}} (\sum_{i=1}^{q_s} b_{i,k}) \cdot \vec{v}_k \in \text{span}(\{\vec{v}_k\}_{k \in \text{cor}})$, and thus it is left to show $b_{i,k} = c_{i,k} \lambda_{i,k} \gamma_{i,k}$ for all $(i, k) \in [q_s] \times \text{hon}$ with $\gamma_{i,k} \neq 0$.

For each $(i, k) \in [q_s] \times \text{hon}$ with $\gamma_{i,k} \neq 0$, since BadRS does not occur, there exists a query $(k, \text{eid}, \mathcal{S}, m, \mathcal{PM}_1)$ to $\mathcal{O}^{\text{Sign}_2}$ such that $a_{i,k} = \text{H}_{\text{non}}(\text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$ (for FROST1 , $a_{i,k} = \text{H}_{\text{non}}(k, \text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$), $c_{i,k} = \text{H}_{\text{sig}}(\text{PK}, m, \tilde{R})$ with $\tilde{R} := \text{CompR}(\text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$, $\lambda_{i,k} = \lambda_k^{\mathcal{S}}$, and $\gamma'_{i,k} = a_{i,k} \gamma_{i,k}$. We now show that when A made the H_{sig} query $(\text{PK}, m^*, \tilde{R}^*)$, both conditions in Step 3 hold:

Condition (3a) holds. Let $T = (\text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$ (resp. $(k, \text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$ for FROST1), and we have $\gamma'_{i,k} = \text{H}_{\text{non}}[T] \gamma_{i,k}$. Also, the query T to H_{non} must be made before A made the H_{sig} query $(\text{PK}, m^*, \tilde{R}^*)$. Otherwise, BadNon would occur since $\text{Flag}_{\text{non}}[\gamma'_{i,k}/\gamma_{i,k} (= \text{H}_{\text{non}}(\text{PK}, \mathcal{S}, m, \mathcal{PM}_1))]$ is set to 1 after the H_{sig} query (see Step 4), which contradicts the fact that $\mathcal{C}$ does not abort. Therefore, the first condition holds.

Condition (3b) holds. Since $\mathcal{A}$ wins the unforgeability game, we have $m \neq m^*$, which implies the second condition holds.

Therefore, from the description of Step 3, $b_{i,k} = \text{H}_{\text{sig}}(\text{PK}, m, \tilde{R}) \lambda_i^{\mathcal{S}} \gamma_{i,k} = c_{i,k} \lambda_{i,k} \gamma_{i,k}$. Since $\mathcal{C}$ simulates $\text{Expt}_{\mathcal{A}, \text{FROST}}^{\text{adp-TS-UF}}$ perfectly except for an abort if BadNon occurs,

$$\text{Adv}_{\mathcal{C}}^{(t_c, t, n)\text{-ldvr}}(\kappa) \geq \Pr[\text{Succ} \wedge \neg(\text{BadRS} \vee \text{Badv})] - \Pr[\text{BadNon}].$$

For each H_{non} query, since the number of values marked by Flag_{non} is at most $2nq$,⁷ the probability that the sampled a collides with one of the values is at most $2nq/p$. (For FROST1 , the probability that one of the sampled a_i collides

⁷ At most n values can be marked for each H_{non} query or H_{sig} query.

with one of the values is at most $2n^2q/p$). Therefore, $\Pr[\text{BadNon}] \leq 2(nq)^2/p$, which concludes the proof of the lemma. $\square$

4.4 Hardness of LDVR

In this section, we provide some basic intuition about the hardness of the LDVR problem. First note that $\{\vec{v}_i\}_{i \in \text{CS}}$ can never be a basis of $\mathbb{Z}_p^{t+1}$ (since $t_c < t + 1$), and therefore it is not clear whether the problem is solvable. (Indeed, we show below that for some settings of the parameters even an unbounded adversary cannot solve it except with negligible probability.) In particular, note that $\vec{v}_0 \notin \text{span}(\{\vec{v}_i\}_{i \in \text{CS}})$ for any $\text{CS} \subset [n]$ of size t_c , and so if we fix such a CS in advance then for any query $c := \mathcal{O}(\vec{\alpha})$ made by $\mathcal{A}$, the probability that $\vec{w} = c \cdot \vec{v}_0 + \sum_{i=0}^n \alpha[i] \cdot \vec{v}_i$ lies in $\text{span}(\{\vec{v}_i\}_{i \in \text{CS}})$ is at most $1/p$. Note, however, that $\mathcal{A}$ can choose CS *after* observing c , and there are $\binom{n}{t_c}$ possibilities for CS. Nevertheless, this already shows that the (t_c, t, n) -LDVR problem is unconditionally hard (for an algorithm making polynomially many queries to its oracle) if $q \binom{n}{t_c} \cdot 2^{-\kappa}$ is negligible. We show additional results about unconditional hardness next, which extend this result to a general corruption threshold $t_c < t$.

Unconditional hardness. We prove unconditional hardness of the (t_c, t, n) -LDVR problem for various settings of t_c, t, n . Specifically, we provide an upper bound on the probability with which even an *unbounded* algorithm $\mathcal{A}$ can solve the LDVR problem, as a function of t_c, t, n and the number of oracle queries q made by $\mathcal{A}$. We also give numerical examples of our bound.

In the following, we define $(n)_k = n \cdot (n - 1) \cdots (n - k + 1)$ for $k \geq 1$, with $(n)_k = 1$ if $k \leq 0$.

Theorem 3 (Unconditional hardness of LDVR problem). *Let $t_c \leq t < n$ be integers. Then, for any algorithm $\mathcal{A}$ making $q = q(\kappa)$ queries to its oracle,*

$$\text{Adv}_{\mathcal{A}}^{(t_c, t, n)\text{-ldvr}}(\kappa) \leq q \cdot \frac{(n)_{2t_c-t}}{(t_c)_{2t_c-t}} \cdot \frac{1}{2^\kappa}.$$

Discussion of the bound. Before proving Theorem 3, we make some observations. First note that the inefficient attack from prior work [CS] can be seen as an algorithm solving the LDVR problem (as discussed in Section 4.1), and is thus subject to our bound. For $t_c = t$, that prior work shows an algorithm that succeeds with probability $\frac{1}{2}$ if $\binom{n}{t} > 2^\kappa$. This implies that our upper bound is tight when $t_c = t$, since then $2t_c - t = t$ and

$$\frac{(n)_t}{(t)_t} = \frac{(n)_t}{t!} = \binom{n}{t}.$$

For $t_c < t$, the algorithm from prior work succeeds with probability lower than our upper bound, and it remains open whether our bound is tight in that range. Also, compared to the trivial upper bound $q \binom{n}{t_c} \cdot 2^{-\kappa}$ mentioned above, the improved upper bound is strictly better since $\binom{n}{t_c} = \frac{(n)_{t_c}}{(t_c)_{t_c}} < \frac{(n)_{2t_c-t}}{(t_c)_{2t_c-t}}$ given

(t, n)	δ					
	0.55	0.60	0.70	0.80	0.90	1.00
(32, 128)	11.77	17.61	35.09	58.85	77.85	100.22
(64, 128)	11.71	23.06	48.54	69.82	95.66	124.17
(96, 128)	13.51	25.95	45.84	64.55	79.24	100.22
(64, 256)	17.79	35.45	76.57	112.48	157.12	203.57
(128, 256)	23.51	50.07	97.52	140.36	188.59	251.67
(192, 256)	27.10	49.67	92.02	129.66	161.01	203.57
(256, 1024)	77.46	154.33	301.98	458.68	619.17	825.63
(512, 1024)	102.08	197.18	384.26	570.84	761.94	1018.67

Table 1. Numerical values from Theorem 3. Selected values of the function $f_{t,n}(t_c = \lfloor \delta \cdot t \rfloor) = \log \left(\frac{\binom{n}{2t_c-t}}{\binom{n}{2t_c-t}} \right)$, where $\lfloor x \rfloor$ denotes rounding x to the nearest integer.

$t_c < t$. Moreover, improvement is significant when t_c is far from t . For example, if $t_c = 2t/3$, the trivial upper bound gives $q^{\frac{\binom{n}{2t/3}}{\binom{n}{2t/3}}} \cdot 2^{-\kappa}$, while the improved bound gives $q^{\frac{\binom{n}{t/3}}{\binom{n}{t/3}}} \cdot 2^{-\kappa}$, which is better by a factor of $\frac{\binom{n-t/3}{t/3}}{\binom{n}{t/3}}$.

Observe also that for $t_c \leq t/2$, the above bound is $q/2^\kappa$; when plugged into Theorem 2, this implies (as implicitly confirmed by Theorem 1) that the LDVR problem is not relevant in that case.

Remark 1 (Comparison with guessing argument). It is also interesting to compare what we get by composing Theorems 2 and 3 with the folklore approach of lifting static to adaptive security via guessing the corruption set. The least favorable comparison is for the case of full corruption $t_c = t$, where our approach results in an additive term of $q \binom{n}{t} / 2^\kappa$ with q corresponding to the number of random-oracle queries made by the attacker. In contrast, employing the guessing strategy results in a multiplicative term of $\binom{n}{t}$. Given current concrete bounds for the static security of FROST (e.g., [BFP21]), this means that if we can prove that the advantage of a static adversary is at most ϵ , we can also show that the advantage of an adaptive one is at most $\binom{n}{t} \epsilon$. The best known ϵ is effectively the bound of Theorem 2 without the LDVR advantage, and once one factors in concrete security bounds for AOMDL (e.g., from the GGM analysis [BFP21]), the final concrete bound would be looser. The improvement is even more important for $t_c < t$, since our bound is smaller than $\binom{n}{t_c} / 2^\kappa$.

Table 1 gives numerical values for $f_{t,n}(t_c = \delta \cdot t) = \log \left(\frac{\binom{n}{2t_c-t}}{\binom{n}{2t_c-t}} \right)$ that provide a quantitative understanding of the bound from Theorem 3. For example, we see that if $n = 128$, $t = t_c = 96$, and $p > 2^{256}$, Theorem 3 gives

$$\text{Adv}_{\mathcal{A}}^{(t_c, t, n)\text{-ldvr}}(\kappa) \leq q \cdot 2^{100.22-256} = \frac{q}{2^{155.78}}.$$

We now prove Theorem 3:

Proof. We assume $\mathcal{A}$ makes exactly q queries, and let p be the prime output by $\mathcal{A}$. For $i \in [q]$, let Win_i be the event that the vector $\vec{w}_i = c_i \cdot \vec{v}_0 + \sum_{j=0}^n \vec{\alpha}_i[j] \cdot \vec{v}_j$ implicitly defined by the i th query $\vec{\alpha}_i$ returning c_i belongs to $\text{Span}(\{\vec{v}_j\}_{j \in S})$ for some $S \subseteq [n]$ with $|S| = t_c$. Note that if the game outputs **true** then Win_i must occur for some $i \in [q]$, and so a union bound implies

$$\text{Adv}_{\mathcal{A}}^{(t_c, t, n)\text{-ldvr}}(\kappa) \leq \sum_{i=1}^q \Pr[\text{Win}_i]. \quad (18)$$

Fix some $i \in [q]$, and write

$$\vec{w}_i = (c_i + \vec{\alpha}_i[0]) \cdot \vec{v}_0 + \vec{u}_i,$$

where $\vec{u}_i = \sum_{j=1}^n \vec{\alpha}_i[j] \cdot \vec{v}_j$. We call a set $S \subseteq [n]$ of size t_c *good* if $\vec{u}_i \in \text{Span}(\{\vec{v}_j\}_{j \in S} \cup \{\vec{v}_0\})$. In particular, for every good set S , there exist coefficients $(\beta_j^S)_{j \in S \cup \{0\}}$ such that $\vec{u}_i = \beta_0^S \vec{v}_0 + \sum_{j \in S} \beta_j^S \cdot \vec{v}_j$.

Claim. Win_i occurs if and only if $c_i + \vec{\alpha}_i[0] + \beta_0^S = 0$ for some good set S .

Proof. If $c_i + \vec{\alpha}_i[0] + \beta_0^S = 0$ for some good set S , then $\vec{w}_i \in \text{Span}(\{\vec{v}_j\}_{j \in S})$ (as the $\vec{v}_0$ component cancels out) and thus Win_i occurs. Conversely, if Win_i occurs then there exists $S \subseteq [n]$ with $|S| = t_c$ such that $\vec{w}_i \in \text{Span}(\{\vec{v}_j\}_{j \in S})$. We claim that S must be good. To see this, let $S' \supseteq S$ be smallest such that $\vec{u}_i \in \text{Span}(\{\vec{v}_j\}_{j \in S'} \cup \{\vec{v}_0\})$. If $S' \neq S$, then $\vec{w}_i$ cannot be in $\text{Span}(\{\vec{v}_j\}_{j \in S})$, as it would have at least one non-zero component aligned with $\vec{v}_j$ for $j \notin S \cup \{0\}$. Further, if $c_i + \vec{\alpha}_i[0] + \beta_0^S \neq 0$, it would also have a component aligned with $\vec{v}_0$, and thus not be in $\text{Span}(\{\vec{v}_j\}_{j \in S})$. $\square$

Let **Good** be the collection of good sets. Because **Good** is completely determined by $\vec{\alpha}_i$, and c_i is uniform and independent of $\vec{\alpha}_i$, we have

$$\Pr[\text{Win}_i] = \Pr[\exists S \in \text{Good} : c_i + \vec{\alpha}_i[0] + \beta_0^S = 0] \leq \frac{|\text{Good}|}{p}. \quad (19)$$

It remains to upper bound $|\text{Good}|$. To this end, fix distinct $S, S' \in \text{Good}$. Then, we can write $\vec{w}_i$ in two different ways, namely

$$(c_i + \vec{\alpha}_i[0] + \beta_0^S) \vec{v}_0 + \sum_{j \in S} \beta_j^S \cdot \vec{v}_j = \vec{w}_i = (c_i + \vec{\alpha}_i[0] + \beta_0^{S'}) \vec{v}_0 + \sum_{j \in S'} \beta_j^{S'} \cdot \vec{v}_j$$

which implies in particular that

$$(\beta_0^S - \beta_0^{S'}) \vec{v}_0 + \sum_{j \in S} \beta_j^S \cdot \vec{v}_j - \sum_{j \in S'} \beta_j^{S'} \cdot \vec{v}_j = \vec{0}.$$

Thus $|S \cup S'| \geq t + 1$, for otherwise we would have expressed $\vec{0}$ as a non-zero linear combination of the vectors $\{\vec{v}_j\}_{j \in S \cup S'} \cup \{\vec{v}_0\}$, contradicting their linear

independence. Therefore, if we define Good' to be the set of subsets $S \subseteq [n]$ of size t_c such that $|S \cup S'| \geq t + 1$ for any distinct $S, S' \in \text{Good}'$, we have $|\text{Good}| \leq |\text{Good}'|$. It is thus enough to upper bound the latter.

If $t_c \leq t/2$, then Good' includes at most one set, which proves the claimed upper bound in that case. To obtain a bound for $t_c > t/2$, we assign every set $S \in \text{Good}'$ to its characteristic vector $\vec{c}_S \in \{0, 1\}^n$, which has weight exactly t_c , to build a code $\mathcal{C} = \{\vec{c}_S : S \in \text{Good}'\} \subseteq \{0, 1\}^n$. We observe that for any two $S, S' \in \text{Good}'$, the Hamming distance of $\vec{c}_S$ and $\vec{c}_{S'}$ is

$$\begin{aligned} \Delta(\vec{c}_S, \vec{c}_{S'}) &= |S \oplus S'| = |S| + |S'| - 2|S \cap S'| \\ &= |S| + |S'| - 2(|S| + |S'| - |S \cup S'|) \\ &= 2|S \cup S'| - |S| - |S'| \geq 2(t + 1 - t_c), \end{aligned}$$

where $S \oplus S'$ denotes symmetric difference of the two sets. By invoking the Johnson bound (Lemma 2) with $e = t + 1 - t_c$, we obtain the desired bound for $|\text{Good}'| = |\mathcal{C}|$, since $w - e + 1 = t_c - t - 1 + t_c + 1 = 2t_c - t$. Combining Equations (18) and (19), and using the fact that $p \geq 2^\kappa$, concludes the proof. $\square$

For completeness, we provide a proof of the following in Appendix B.

Lemma 2 (Johnson Bound). *For any integers $e \leq w \leq n$, let $\mathcal{C} \subseteq \{0, 1\}^n$ be a binary code such that every codeword $\vec{c} \in \mathcal{C}$ has weight w , and any two distinct $\vec{c}, \vec{c}' \in \mathcal{C}$ have Hamming distance $\Delta(\vec{c}, \vec{c}') \geq 2e$. Then,*

$$|\mathcal{C}| \leq \prod_{i=0}^{w-e} \frac{n-i}{w-i} = \frac{(n)_{w-e+1}}{(w)_{w-e+1}}.$$

Computational hardness. We conjecture that the LDVR problem is computationally hard for any $t_c \leq t < n$. In Section 4.5 we show that one natural algorithmic strategy for solving the problem results in a modular subset-sum problem which is hard for current algorithms. Of course, further cryptanalysis of the LDVR problem is needed.

4.5 Cryptanalysis of the LDVR Problem

In order to support a plausible conjecture that LDVR is hard for any choice of $t_c \leq t < n$, we describe here a cryptanalytic attack strategy for the LDVR problem which targets the maximum corruption case $t_c = t$, under the assumption that $n \geq t + 2$. Clearly, $t_c = t$ is the easiest case to attack, since an attack for a lower t_c would also apply. One of the proposed attacks below runs in time $\tilde{O}(2^{\kappa/2})$ under reasonable conjectures, but it appears hard to improve upon this approach.

The attack template. Fix a prime $2^\kappa < p < 2^{\kappa+1}$. The basic version of the attack queries a single vector $\alpha = (0, 1, 0, \dots, 0)$ to $\mathcal{O}$, which is then associated with a single uniform challenge $c \in \mathbb{Z}_p$. We let $\vec{w} = c \cdot \vec{v}_0 + \vec{v}_1 \in \mathbb{Z}_p^{t+1}$, and we aim to find a set $\text{CS} \subseteq [2..n]$ of size $|\text{CS}| = t_c$ such that $\vec{w} \in \text{Span}(\{\vec{v}_i\}_{i \in \text{CS}})$.

We can now proceed as follows to find such a set CS. First off, we notice that $\{\vec{v}_i\}_{i \in \text{CS} \cup \{0\}}$ is a basis of $\mathbb{Z}_p^{t+1}$ for any CS with $|\text{CS}| = t_c$, and thus we can find coefficients $(\alpha_i^{\text{CS}})_{i \in \text{CS} \cup \{0\}}$ such that

$$\vec{v}_1 = \sum_{i \in \text{CS} \cup \{0\}} \alpha_i^{\text{CS}} \vec{v}_i .$$

We note that the adversary wins if it can find CS such that $\alpha_0^{\text{CS}} = -c = p - c$. To this end, the following claim gives an explicit formula for α_0^{CS} .

Claim. $\alpha_0^{\text{CS}} = \prod_{j \in \text{CS}} (1 - j^{-1})$

Proof. Let $2 \leq j_1 < j_2 < \dots < j_{t_c} \leq n$ be the elements of CS. Then, we want to solve the system of equations

$$(\alpha_0^{\text{CS}}, \alpha_{j_1}^{\text{CS}}, \dots, \alpha_{j_{t_c}}^{\text{CS}}) \underbrace{\begin{bmatrix} \vec{v}_0 \\ \vec{v}_{j_1} \\ \vdots \\ \vec{v}_{j_{t_c}} \end{bmatrix}}_{=: \vec{V}_{\text{CS}}} = \vec{v}_1 = (1, 1, \dots, 1) .$$

Now, let $\vec{a} \in \mathbb{Z}_p^{t+1}$ be a (row) vector such that its transpose $\vec{a}^\top$ is the first column of the inverse $\vec{V}_{\text{CS}}^{-1}$ of $\vec{V}_{\text{CS}}$. It can be thought of as defining the coefficients of a polynomial $a(X)$ of degree t such that $a(0) = \vec{v}_0 \vec{a}^\top = 1$, and $a(j) = \vec{v}_j \vec{a}^\top = 0$ for all $j \in \text{CS}$. By Lagrange interpolation, this polynomial is uniquely defined as

$$a(X) = 1 \cdot \prod_{j \in S} \frac{X - j}{0 - j} = \prod_{j \in S} (1 - j^{-1} X) .$$

The claim follows by observing that $\alpha_0^{\text{CS}} = \vec{v}_1 \vec{a}^\top = a(1)$. □

Attacks. We only give some high-level ideas about attack strategies that follow this template.

A first attack, given a random $-c$, attempts to find a subset $\text{CS} \subseteq [2..n]$ with size $|\text{CS}| = t$ such that $\alpha_0^{\text{CS}} = \prod_{j \in \text{CS}} (1 - j^{-1}) = -c$. This can be done by using dynamic programming in time $\Theta(n \cdot t \cdot p)$, nothing that in general $n \cdot t$ is much smaller than p .

It is not clear how to prove a formal lower bound on the success probability of this attack. However, under the assumption that $\binom{n-1}{t} > p$, it is plausible to conjecture that the set of potential values α_0^{CS} covers a constant fraction of $\mathbb{Z}_p$, and thus the success probability is constant.

A better attack strategy exploits multiple targets. In particular, assume the adversary queries $\alpha = (0, 1, 0, \dots, 0)$ a total of $q = \sqrt{p}$ times to $\mathcal{O}$ (recall that repeat queries *are* allowed); each query is then associated with a uniform challenge

$c_i \in \mathbb{Z}_p$ defining a vector $\vec{w}_i = c_i \cdot \vec{v}_0 + \vec{v}_1 \in \mathbb{Z}_p^{t+1}$. Our goal now is to find CS such that $\alpha_0^{\text{CS}} = \prod_{j \in \text{CS}} (1 - j^{-1}) \in \{c_1, \dots, c_q\}$.

We can then use the following simple strategy of picking random sets $\text{CS}_1, \text{CS}_2, \dots, \text{CS}_q$, and if $\alpha_0^{\text{CS}_j} \in \{c_1, \dots, c_q\}$ for some $j \in [1..q]$, we have succeeded. This strategy can be implemented in time $\tilde{O}(\sqrt{p})$, where we are hiding small polylogarithmic terms in p . We note that the probability that each CS_i satisfies $\alpha_0^{\text{CS}_j} \in \{c_1, \dots, c_q\}$ is approximately q/p , and the expected number of j 's for which this is true is $q^2/p = 1$.

Of course, to formally validate this attack, we would have to prove that the $\alpha_0^{\text{CS}_j}$'s are sufficiently independent, but we take this simple attack as strong validation that an attack taking time $2^{\kappa/2}$ exists.

Acknowledgments

Research of the last two authors was partially supported by NSF grants CNS-2026774, CNS-2154174, and CNS-2426905, a gift from Microsoft, and a Stellar Development Foundation Academic Research Award.

References

- ADN06. Jesús F. Almansa, Ivan Damgård, and Jesper Buus Nielsen. Simplified threshold RSA with adaptive and proactive security. In Serge Vaudenay, editor, *EUROCRYPT 2006, 25th Annual International Conference on the Theory and Applications of Cryptographic Techniques, St. Petersburg, Russia, May 28 - June 1, 2006*, volume 4004 of *LNCS*, pages 593–611. Springer, 2006.
- AF04. Masayuki Abe and Serge Fehr. Adaptively secure feldman VSS and applications to universally-composable threshold cryptography. In Matthew K. Franklin, editor, *Advances in Cryptology - CRYPTO 2004, 24th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 2004, Proceedings*, volume 3152 of *LNCS*, pages 317–334. Springer, 2004.
- BBC⁺25. Michele Battagliola, Giacomo Borin, Giovanni Di Crescenzo, Alessio Meneghetti, and Edoardo Persichetti. Enhancing threshold group action signature schemes: Adaptive security and scalability improvements. In Ruben Niederhagen and Markku-Juhani O. Saarinen, editors, *Post-Quantum Cryptography - 16th International Workshop, PQCrypto 2025, Taipei, Taiwan, April 8-10, 2025, Proceedings, Part I*, volume 15577 of *Lecture Notes in Computer Science*, pages 129–161. Springer, 2025.
- BCK⁺22. Mihir Bellare, Elizabeth Crites, Chelsea Komlo, Mary Maller, Stefano Tessaro, and Chenzhi Zhu. Better than advertised security for non-interactive threshold signatures. In *Crypto 2022*, LNCS, 2022.
- BD22. Luís Brandão and Michael Davidson. Notes on threshold EdDSA/Schnorr signatures. <https://nvlpubs.nist.gov/nistpubs/ir/2022/NIST.IR.8214B.ipd.pdf>, 2022.

- BDLR24. Renas Bacho, Sourav Das, Julian Loss, and Ling Ren. Glacius: Threshold Schnorr signatures from DDH with full adaptive security, 2024. Available at <https://eprint.iacr.org/2024/1628>.
- BFP21. Balthazar Bauer, Georg Fuchsbauer, and Antoine Plouviez. The one-more discrete logarithm assumption in the generic group model. In Mehdi Tibouchi and Huaxiong Wang, editors, *Asiacrypt 2021*, volume 13093 of *LNCS*, pages 587–617. Springer, 2021.
- BL22. Renas Bacho and Julian Loss. On the adaptive security of the threshold BLS signature scheme. *IACR Cryptol. ePrint Arch.*, page 534, 2022.
- BLSW24. Renas Bacho, Julian Loss, Gilad Stern, and Benedikt Wagner. HARTS: High-threshold, adaptively secure, and robust threshold Schnorr signatures, 2024. Available at <https://eprint.iacr.org/2024/280>.
- BLT⁺23. Renas Bacho, Julian Loss, Stefano Tessaro, Benedikt Wagner, and Chenzhi Zhu. Twinkle: Threshold signatures from DDH with full adaptive security. *IACR Cryptol. ePrint Arch.*, page 1482, 2023.
- BN06. Mihir Bellare and Gregory Neven. Multi-signatures in the plain public-key model and a general forking lemma. In Ari Juels, Rebecca N. Wright, and Sabrina De Capitani di Vimercati, editors, *CCS 2006*, pages 390–399. ACM, 2006.
- BP02. Mihir Bellare and Adriana Palacio. GQ and Schnorr identification schemes: Proofs of security against impersonation under active and concurrent attacks. In Moti Yung, editor, *Crypto 2002*, volume 2442 of *LNCS*, pages 162–177. Springer, 2002.
- BP25. Luís Brandão and Rene Peralta. NIST first call for multi-party threshold schemes. <https://nvlpubs.nist.gov/nistpubs/ir/2025/NIST.IR.8214C.2pd.pdf>, 2025.
- BS07. Mihir Bellare and Sarah Shoup. Two-tier signatures, strongly unforgeable signatures, and Fiat-Shamir without random oracles. In Tatsuaki Okamoto and Xiaoyun Wang, editors, *PKC 2007*, volume 4450 of *LNCS*, pages 201–216. Springer, 2007.
- BTZ22. Mihir Bellare, Stefano Tessaro, and Chenzhi Zhu. Stronger security for non-interactive threshold signatures, 2022. Available at <https://eprint.iacr.org/2022/833>.
- CCL⁺23. Guilhem Castagnos, Dario Catalano, Fabien Laguillaumie, Federico Savasta, and Ida Tucker. Bandwidth-efficient threshold EC-DSA revisited: On-line/offline extensions, identifiable aborts proactive and adaptive security. *Theor. Comput. Sci.*, 939:78–104, 2023.
- CG23. Deirdre Connolly and Conrado Gouvea. Speeding up FROST with multi-scalar multiplication, 2023. Available at <https://zfnd.org/speeding-up-frost-with-multi-scalar-multiplication>.
- CGG⁺20. Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. UC non-interactive, proactive, threshold ECDSA with identifiable aborts. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*, pages 1769–1787. ACM, 2020.
- CGRS23. Hien Chu, Paul Gerhart, Tim Ruffing, and Dominique Schröder. Practical Schnorr threshold signatures without the algebraic group model. In Helena Handschuh and Anna Lysyanskaya, editors, *Crypto 2023*, volume 14081 of *LNCS*, pages 743–773. Springer, 2023.

- CKGW22. Deirdre Connolly, Chelsea Komlo, Ian Goldberg, and Chris Wood. Two-round threshold Schnorr signatures with FROST, 2022. Available at <https://datatracker.ietf.org/doc/draft-irtf-cfrg-frost>.
- CKM21. Elizabeth C. Crites, Chelsea Komlo, and Mary Maller. How to prove Schnorr assuming Schnorr: Security of multi- and threshold signatures, 2021. Available at <https://eprint.iacr.org/2021/1375>.
- CKM23. Elizabeth C. Crites, Chelsea Komlo, and Mary Maller. Fully adaptive Schnorr threshold signatures. In Helena Handschuh and Anna Lysyanskaya, editors, *Crypto 2023*, volume 14081 of *LNCS*, pages 678–709. Springer, 2023.
- CKM25. Elizabeth Crites, Chelsea Komlo, and Mary Maller. On the adaptive security of key-unique threshold signatures. Cryptology ePrint Archive, Paper 2025/943, 2025.
- CS. Elizabeth Crites and Alistair Stewart. A plausible attack on the adaptive security of threshold Schnorr signatures. *To appear in Crypto 2025*.
- DR23. Sourav Das and Ling Ren. Adaptively secure BLS threshold signatures from DDH and co-cdh. *IACR Cryptol. ePrint Arch.*, page 1553, 2023.
- Fis05. Marc Fischlin. Communication-efficient non-interactive proofs of knowledge with online extractors. In Victor Shoup, editor, *CRYPTO 2005, Santa Barbara, California, USA, August 14-18, 2005*, volume 3621 of *LNCS*, pages 152–168. Springer, 2005.
- FKL18. Georg Fuchsbauer, Eike Kiltz, and Julian Loss. The algebraic group model and its applications. In Hovav Shacham and Alexandra Boldyreva, editors, *Crypto 2018*, volume 10992 of *LNCS*, pages 33–62. Springer, 2018.
- FPS20. Georg Fuchsbauer, Antoine Plouviez, and Yannick Seurin. Blind Schnorr signatures and signed ElGamal encryption in the algebraic group model. In Anne Canteaut and Yuval Ishai, editors, *Eurocrypt 2020*, volume 12106 of *LNCS*, pages 63–95. Springer, 2020.
- GKC22. Conrado Gouvea, Chelsea Komlo, and Deirdre Connolly. FROST for spend authorization multisignatures, 2022. Available at <https://zips.z.cash/zip-0312>.
- KG20. Chelsea Komlo and Ian Goldberg. FROST: Flexible round-optimized Schnorr threshold signatures. In Orr Dunkelman, Michael J. Jacobson Jr., and Colin O’Flynn, editors, *SAC 2020*, volume 12804 of *LNCS*, pages 34–65. Springer, 2020.
- KG24. Chelsea Komlo and Ian Goldberg. Arctic: Lightweight and stateless threshold Schnorr signatures. Cryptology ePrint Archive, Paper 2024/466, 2024.
- KG24. Brian Koziel, S. Dov Gordon, and Craig Gentry. Fast two-party threshold ECDSA with proactive security. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, pages 1567–1580. ACM, 2024.
- KRT24. Shuichi Katsumata, Michael Reichle, and Kaoru Takemure. Adaptively secure 5-round threshold signatures from MLWE/MSIS and DL with rewinding. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology—Crypto 2024, Part VII*, volume 14926 of *LNCS*, pages 459–491. Springer, 2024.
- Lin24. Yehuda Lindell. Simple three-round multiparty Schnorr signing with full simulatability. *IACR Communications in Cryptology*, 1(1), 2024.
- Mak22. Nikolaos Makriyannis. On the classic protocol for MPC Schnorr signatures. Cryptology ePrint Archive, 2022. Available at <https://eprint.iacr.org/2022/1332>.

- NKDM03. Antonio Nicolosi, Maxwell N. Krohn, Yevgeniy Dodis, and David Mazières. Proactive two-party signatures for user authentication. In *NDSS 2003*. The Internet Society, 2003.
- NRS21. Jonas Nick, Tim Ruffing, and Yannick Seurin. MuSig2: Simple two-round Schnorr multi-signatures. In Tal Malkin and Chris Peikert, editors, *Crypto 2021*, volume 12825 of *LNCS*, pages 189–221. Springer, 2021.
- PS00. David Pointcheval and Jacques Stern. Security arguments for digital signatures and blind signatures. *J. Cryptol.*, 13(3):361–396, 2000.
- RRJ⁺22. Tim Ruffing, Viktoria Ronge, Elliott Jin, Jonas Schneider-Bensch, and Dominique Schröder. ROAST: Robust asynchronous schnorr threshold signatures. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *CCS 2022*, pages 2551–2564. ACM, 2022.
- Sch91. Claus-Peter Schnorr. Efficient signature generation by smart cards. *J. Cryptol.*, 4(3):161–174, 1991.
- Sha79. Adi Shamir. How to share a secret. *Comm. ACM*, 22(11):612–613, 1979.
- TZ23. Stefano Tessaro and Chenzhi Zhu. Threshold and multi-signature schemes from linear hash functions. In Carmit Hazay and Martijn Stam, editors, *Eurocrypt 2023*, volume 14008 of *LNCS*, pages 628–658. Springer, 2023.

A Proof of Adaptive Security for $t/2$ Corruptions

We now prove Theorem 1. Our reduction makes use of rewinding. We give the general forking algorithm by Bellare and Neven [BN06] in Figure 6, and general forking lemma next.

Lemma 3 (General Forking Lemma [BN06]). *Let $q \in \mathbb{N}$ and H be a finite, non-empty set. Let IG be a randomized algorithm that generates an input X . Let $\mathcal{C}$ be a randomized algorithm that takes as input X and $h_1, \dots, h_q \in H$ and random coins ρ and outputs an index $I \in [0..q]$ and side output out . Let $\text{accept}(\mathcal{C})$ be the probability that $\mathcal{C}$ outputs an index $I \neq 0$ in the following experiment:*

$$\text{accept}(\mathcal{C}) := \Pr[X \leftarrow \text{IG}; h_1, \dots, h_q \leftarrow H; (I, \text{out}) := \mathcal{C}(X, (h_1, \dots, h_q); \rho) : I \neq 0]$$

Let the forking algorithm $\text{Fork}^{\mathcal{C}}(X)$ associated to $\mathcal{C}$ be the randomized algorithm that takes as input X and proceeds as in Figure 6. Let

$$\text{accept}(\text{Fork}^{\mathcal{C}}) := \Pr[X \leftarrow \text{IG}; \text{output} \leftarrow \text{Fork}^{\mathcal{C}}(X) : \text{output} \neq \perp]$$

Then, $\text{accept}(\text{Fork}^{\mathcal{C}})$ is bounded by

$$\text{accept}(\text{Fork}^{\mathcal{C}}) \geq \text{accept}(\mathcal{C}) \cdot \left(\frac{\text{accept}(\mathcal{C})}{q} - \frac{1}{|H|} \right). \quad (20)$$

or, alternatively,

$$\text{accept}(\mathcal{C}) \leq \frac{q}{|H|} + \sqrt{q \cdot \text{accept}(\text{Fork}^{\mathcal{C}})} \quad (21)$$

We now prove Theorem 1.

Algorithm $\text{Fork}^{\mathcal{C}}(X)$ Pick random coins ρ for $\mathcal{C}$. $h_1, \dots, h_q \leftarrow_{\\$} H$ $(I, \text{out}) := \mathcal{C}(X, (h_1, \dots, h_q); \rho)$ return $\perp$ if $I = 0$ $h'_1, \dots, h'_q \leftarrow_{\\$} H$ $(I', \text{out}') := \mathcal{C}(X, (h_1, \dots, h_{I-1}, h'_1, \dots, h'_q); \rho)$ return $\perp$ if $I \neq I' \vee h_I = h'_I$ return $\text{output} := (I, \text{out}, \text{out}')$
--

Fig. 6. The forking algorithm $\text{Fork}^{\mathcal{C}}(X)$ associated to an algorithm $\mathcal{C}$ and input X .

Proof. Let $\mathcal{A}$ be a PPT adversary attempting to break the adaptive unforgeability of FROST (Fig. 3) that makes up to q_h queries to H_{non} and H_{sig} , and q_s queries to $\mathcal{O}^{\text{Sign}_1}$. $\mathcal{A}$ is allowed to make $t/2$ adaptive corruptions.

We assume, without loss of generality, that $\mathcal{A}$ makes a query to H_{sig} on its forgery $(\text{PK}, m^*, \tilde{R}^*)$ and that it always queries $H_{\text{non}}(\text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$ (resp. $H_{\text{non}}(k, \text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$ for FROST1) before querying $\mathcal{O}^{\text{Sign}_1}(k, \mathcal{S}, m, \mathcal{PM}_1)$ for some $k \in \mathcal{S}$. This adds $q_s + 1$ additional random-oracle queries. Then, let $q = q_h + q_s + 1$.

We define an algorithm $\mathcal{C}$ that simulates the adaptive unforgeability game. $\mathcal{C}$ is given an oracle $\mathcal{O}^{\text{dl}}$, which it can query on a group element, and receive its discrete logarithm. $\mathcal{C}$ can query this oracle up to $2q_s + t$ times.

Setup. $\mathcal{C}$ accepts as input an instance X , which is a tuple consisting of the group description $(\mathbb{G}, p, g)$ and a set of $2q_s + t + 1$ AOMDL challenges $(X_0, \tilde{X}_1, \dots, \tilde{X}_t, X_1, X'_1, \dots, X_{q_s}, X'_{q_s})$. In addition, $\mathcal{C}$ accepts as input a set of $q = q_h + q_s + 1$ values $\{h_1, \dots, h_q\}$, as in Figure 6, which it will use to program its random-oracle responses.

Next, $\mathcal{C}$ checks for collisions $h_i = h_j$ for $i, j \in [q], i \neq j$. If so, $\mathcal{C}$ aborts. We refer to this bad event as **ROCollision**.

Otherwise, $\mathcal{C}$ initializes the following values:

1. tables $\mathbf{Q}_{\text{non}} := \emptyset, \mathbf{Q}_{\text{sig}} := \emptyset$, to store random-oracle queries to $H_{\text{non}}, H_{\text{sig}}$ and corresponding simulated responses;
2. table $\mathbf{Q}_m := \emptyset$, to store the set of messages queried by $\mathcal{A}$ to $\mathcal{O}^{\text{Sign}_2}$;
3. table $\mathbf{Q}_{\text{st}} := \emptyset$ of participants' states for all signing sessions initiated by $\mathcal{A}$;
4. table $\mathbf{Q}_{\text{used}} := \emptyset$ to track information associated with each AOMDL pair (X_i, X'_i) used in an $\mathcal{O}^{\text{Sign}_2}$ query;
5. counter $\text{ctr} := 0$, to iterate through pairs in the AOMDL challenges;
6. counter $\text{ctr}_{\text{RO}} := 0$, to iterate through random-oracle queries;
7. counter $\text{ctr}_k := 0$, to count the number of first-round signing queries for each party k .

$\mathcal{C}$ then picks random coins ρ and runs $\mathcal{A}(\text{par}; \rho)$ once on the public parameters $\text{par} = (\mathbb{G}, p, g)$ and randomness ρ .

Simulating KeyGen. To simulate key generation where $\mathcal{C}$ is the trusted dealer, $\mathcal{C}$ uses the AOMDL challenges $(X_0, \hat{X}_1, \dots, \hat{X}_t)$ to define a polynomial $f(Z) = x_0 + \hat{x}_1 Z + \dots + \hat{x}_t Z^t$ “in the exponent” (see Section 2) by deriving each party’s public key as:

$$\text{PK}_i = X_0 \hat{X}_1^i \cdots \hat{X}_t^{i^t} \quad (22)$$

for all $i \in [n]$, where implicitly $\text{sk}_i = f(i)$. The threshold public key is $\text{PK} := X_0$ with corresponding secret key $\text{sk} = x_0 = f(0)$. $\mathcal{C}$ then runs $\mathcal{A}((\mathbb{G}, g, p), \text{PK}, \{\text{PK}_i\}_{i \in [n]})$ with access to the signing and random oracles simulated as follows.

Simulating Random Oracles. When $\mathcal{A}$ queries H_{non} or H_{sig} , $\mathcal{C}$ simulates the response by lazy sampling, as follows.

H_{non} : When $\mathcal{A}$ queries H_{non} on T , for FROST1, $\mathcal{C}$ parses $T = (k, \text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$, and for FROST2 and FROST3, $\mathcal{C}$ parses $T = (\text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$. (Here w.l.o.g. we assume T can be parsed correctly and it is for PK , since otherwise the query on a malformed T would not help the adversary to win the game.) Then, $\mathcal{C}$ checks if $\text{Q}_{\text{non}}[T] = \perp$. If not, $\mathcal{C}$ returns $\text{Q}_{\text{non}}[T]$. Otherwise, $\mathcal{C}$ increments $\text{ctr}_{\text{RO}} := \text{ctr}_{\text{RO}} + 1$, and, for FROST1, $\mathcal{C}$ does the following:

1. For all $i \in \mathcal{S}$, $\mathcal{C}$ samples $a_i := h_{(n+1)\text{ctr}_{\text{RO}}-i}$ and stores $\text{Q}_{\text{non}}[(\text{PK}, \mathcal{S}, m, \{(\ell, R_\ell, S_\ell)\}_{\ell \in \mathcal{S}}, i)] = a_i$.
2. Then, $\mathcal{C}$ derives the aggregate $\tilde{R}$ from T (i.e., $\tilde{R} := \prod_{i \in \mathcal{S}} R_i S_i^{a_i}$) and queries H_{sig} on $(\text{PK}, m, \tilde{R})$.
3. Finally, $\mathcal{C}$ returns a_k .

For FROST2 and FROST3, $\mathcal{C}$ does the following:

1. $\mathcal{C}$ sets $a := h_{2\text{ctr}_{\text{RO}}-1}$ and stores $\text{Q}_{\text{non}}[T] = a$.
2. $\mathcal{C}$ derives the aggregate $\tilde{R}$ from T . (For FROST2, it means that $\mathcal{C}$ parses $\mathcal{PM}_1 = \{(i, R_i, S_i)\}_{i \in \mathcal{S}}$ and computes $\tilde{R} := \prod_{\ell \in \mathcal{S}} R_\ell S_\ell^a$. For FROST3, it means that $\mathcal{C}$ parses $\mathcal{PM}_1 = (\hat{R}, \hat{S})$ and computes $\tilde{R} := \hat{R} \hat{S}^a$.)
3. If $\text{Q}_{\text{sig}}[(\text{PK}, m, \tilde{R})] = \perp$, $\mathcal{C}$ sets $c := h_{2\text{ctr}_{\text{RO}}}$ and stores $\text{Q}_{\text{sig}}[(\text{PK}, m, \tilde{R})] = c$.
4. Finally, $\mathcal{C}$ returns a .

H_{sig} : When $\mathcal{A}$ queries H_{sig} on (PK, m, R) , $\mathcal{C}$ checks if $\text{Q}_{\text{sig}}[(\text{PK}, m, R)] = \perp$. (Similar to H_{non} , here we only consider queries for PK , since otherwise the query would not help the adversary to win the game.) If not, $\mathcal{C}$ samples $c \leftarrow \mathbb{Z}_p$, stores $\text{Q}_{\text{sig}}[(\text{PK}, m, R)] = c$, and returns c . Otherwise, $\mathcal{C}$ increments $\text{ctr}_{\text{RO}} := \text{ctr}_{\text{RO}} + 1$, sets $c := h_{2\text{ctr}_{\text{RO}}}$, (resp. $h_{(n+1)\text{ctr}_{\text{RO}}}$ for FROST1) stores $\text{Q}_{\text{sig}}[(\text{PK}, m, R)] = c$, and returns c .

Simulating Signing Oracles. $\mathcal{C}$ handles $\mathcal{A}$ ’s signing queries as follows.

Round 1 ($\mathcal{O}^{\text{Sign}}(k)$): In the first round of signing, parties form nonce commitments R_i, S_i . When $\mathcal{A}$ queries $\mathcal{O}^{\text{Sign}_1}$ for participant identifier k , $\mathcal{C}$ first checks that $k \in \text{hon}$. If so, then $\mathcal{C}$ does the following:

1. Sets $\text{eid} := \text{ctr}_k$ and increments $\text{ctr}_k := \text{ctr}_k + 1$.
2. Sets $(R_k, S_k) := (X_{\text{ctr}}, X'_{\text{ctr}})$ and increments $\text{ctr} := \text{ctr} + 1$. $\mathcal{C}$ checks if there exists $T = (\text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$ (resp. $T = (k, \text{PK}, \mathcal{S}, m, \mathcal{PM}_1)$ for FROST1) in $\mathbf{Q}_{\text{non}}[T]$ such that $(X_{\text{ctr}}, X'_{\text{ctr}}) = (R_k, S_k)$. If so, $\mathcal{C}$ aborts. We refer to this bad event as **NonceCollision**.
3. Sets $\text{st}_{k,\text{eid},1} := (R_k, \perp, S_k, \perp, \perp)$.
4. Updates $\mathbf{Q}_{\text{st}}[k, \text{eid}, 1] := \text{st}_{k,\text{eid},1}$.
5. Finally, $\mathcal{C}$ returns (R_k, S_k) to $\mathcal{A}$.

Round 2 ($\mathcal{O}^{\text{Sign}_2}(k, \text{eid}, \mathcal{S}, m, \mathcal{PM}_1)$): In the second round of signing, each party in the signing set $\mathcal{S}$ takes as input the first-round nonce commitments $\mathcal{PM}_1$ and outputs is partial signature share z_i .

When $\mathcal{A}$ queries $\mathcal{O}^{\text{Sign}_2}$ on $(k, \text{eid}, \mathcal{S}, m, \mathcal{PM}_1)$ for $k \in \text{hon}$, $\mathcal{C}$ first performs the checks in $\mathcal{O}^{\text{Sign}_2}$ (Fig. 3) and Sign_2 (Fig. 4). If they pass, then, in particular $\text{st}_{k,\text{eid},1} = (R_k, \perp, S_k, \perp, \perp)$. $\mathcal{C}$ then does the following:

1. $\mathcal{C}$ updates $\mathbf{Q}_m := \mathbf{Q}_m \cup \{m\}$.
2. $\mathcal{C}$ follows the Sign_2 algorithm (Fig. 4) to obtain a and c . (For FROST1, a here refers to a_k .) Note that $\mathcal{C}$ can internally query $\mathbf{H}_{\text{non}}$ and $\mathbf{H}_{\text{sig}}$ as needed.
3. $\mathcal{C}$ then sets $Z_k := R_k S_k^a \cdot \text{PK}_k^{c\lambda_k}$, i.e., $Z_k = X_{i'}(X'_{i'})^a X_0^{c\lambda_k} \hat{X}_1^{c\lambda_k k} \dots \hat{X}_t^{c\lambda_k k^t}$ for some $i' \in [q_s]$. $\mathcal{C}$ queries $\mathcal{O}^{\text{dl}}$ on Z_k with representation $\alpha = 0$ and (as in Fig. 1):

$$(\beta_i)_{i=1}^{2q_s+t+1} = (c\lambda_k, c\lambda_k k, \dots, c\lambda_k k^t, 0, \dots, 0, 1, a, 0, \dots, 0),$$

where 1 is in position $t + 2i'$, and receives z_k .

4. $\mathcal{C}$ sets $\text{st}_{k,\text{eid},2} := z_k$.
5. $\mathcal{C}$ adds $((R_k, S_k), (i', z_k, \eta, a, c\lambda_k), \text{query})$ to $\mathbf{Q}_{\text{used}}$, where $\text{query} = (k, \text{eid}, \mathcal{S}, m, \mathcal{PM}_1)$ and η is the index such that $a = h_{2\eta-1}$.
6. Finally, $\mathcal{C}$ returns z_k to $\mathcal{A}$.

Simulating Corruption Queries $\mathcal{O}^{\text{Corrupt}}(k)$: $\mathcal{A}$ may at any time corrupt an honest party k by querying $\mathcal{O}^{\text{Corrupt}}(k)$. Upon receiving a corruption query, $\mathcal{C}$ first checks that $k \in \text{hon}$ and that $|\text{cor}| < t$, returning $\perp$ if not. Otherwise, $\mathcal{C}$ sets $\text{cor} := \text{cor} \cup \{k\}$ and $\text{hon} := \text{hon} \setminus \{k\}$.

$\mathcal{C}$ then queries $\mathcal{O}^{\text{dl}}$ on PK_k with representation $\alpha = 0$ and (recalling that $\text{PK}_k = X_0 \hat{X}_1^k \dots \hat{X}_t^{k^t}$ (Eq. (22)) :

$$(\beta_i)_{i=0}^{2q_s+t} = (1, k, \dots, k^t, 0, \dots, 0)$$

and receives sk_k . $\mathcal{C}$ then does the following:

1. If no entry $\mathbf{Q}_{\text{st}}[k, \text{eid}, 1]$ exists for any eid that has occurred, then $\mathcal{C}$ returns sk_k to $\mathcal{A}$.
2. Otherwise, $\mathcal{C}$ does the following.

- (a) For each entry $Q_{\text{st}}[k, \text{eid}, 1] \neq \perp$ where $Q_{\text{st}}[k, \text{eid}, 2] = \perp$, $\mathcal{C}$ queries $\mathcal{O}^{\text{dl}}$ on R_k , which is equal to $X_{i'}$ for some $i' \in [q_s]$, with representation $\alpha = 0$ and $(\beta_i)_{i=0}^{2q_s+t} = (0, \dots, 0, 1, 0, \dots, 0)$, where 1 is in position $t + 2i'$, and receives r_k . $\mathcal{C}$ similarly queries $\mathcal{O}^{\text{dl}}$ on S_k (with representation $\alpha = 0$ and $(\beta_i)_{i=0}^{2q_s+t} = (0, \dots, 0, 1, 0, \dots, 0)$, where 1 is in position $t + 2i' + 1$), and receives s_k . $\mathcal{C}$ then sets $Q_{\text{st}}[k, \text{eid}, 1] = (R_k, r_k, S_k, s_k, \text{sk}_k)$.
- (b) For each entry where $Q_{\text{st}}[k, \text{eid}, 2] \neq \perp$, $\mathcal{C}$ queries $\mathcal{O}^{\text{dl}}$ on S_k , which is equal to $X'_{i'+1}$ for some $i' \in [q_s]$, with representation $\alpha = 0$ and $(\beta_i)_{i=0}^{2q_s+t} = (0, \dots, 0, 1, 0, \dots, 0)$, where 1 is in position $t + i' + 2$, and receives s_k . $\mathcal{C}$ already made an $\mathcal{O}^{\text{dl}}$ query to obtain z_k , so it can compute $r_k = z_k - s_k a - c \lambda_k \text{sk}_k$. (For FROST1, a here refers to a_k .) $\mathcal{C}$ then sets $Q_{\text{st}}[k, \text{eid}, 1] = (R_k, r_k, S_k, s_k, \text{sk}_k)$.

Finally, $\mathcal{C}$ returns sk_k and all state $Q_{\text{st}}[k, \cdot, \cdot]$ for party k to $\mathcal{A}$.

Output. When $\mathcal{A}$ terminates with output (m^*, σ^*) , $\mathcal{C}$ first checks that it is a valid forgery, by checking that $m^* \notin Q_m$ and $\text{Verify}(\text{PK}, m^*, \sigma^*) = 1$. If either check fails, $\mathcal{C}$ returns 0.

Otherwise, $\mathcal{C}$ looks up the index I for $(\text{PK}, m^*, R^*, c^*) \in Q_{\text{sig}}$ corresponding to $\mathcal{A}$'s forgery. It sets $\text{out} := (\sigma^*, \text{aux}^*)$, where aux^* are all variables received or generated by $\mathcal{C}$, and outputs (I, out) .

$\mathcal{C}$'s simulation is perfect when the values in the instance X are sampled uniformly at random. $\mathcal{C}$ aborts with negligible probability in the following cases:

1. **ROCollision:** $\mathcal{C}$ aborts due to ROCollision with probability less than $\frac{q^2}{p}$.
2. **NonceCollision:** $\mathcal{C}$ aborts due to NonceCollision with probability less than $\frac{q^2}{2p}$.

Reduction $\mathcal{B}$ Against AOMDL. Finally, we construct our PPT reduction $\mathcal{B}$ against the AOMDL assumption (Fig. 1). Specifically, we show how $\mathcal{B}$ uses $\text{Fork}^{\mathcal{C}}$ as a subroutine such that:

$$\text{Adv}_{\mathcal{B}}^{t+1\text{-aomdl}}(\kappa) \geq \text{accept}(\text{Fork}^{\mathcal{C}}(X)) - 2 \Pr[\text{ROCollision}] - 2 \Pr[\text{NonceCollision}] \quad (23)$$

where X is the instance given by the AOMDL challenger.

Initialization. $\mathcal{B}$ is initialized by the AOMDL challenger with input the group description $(\mathbb{G}, p, g)$ and $2q_s + t + 1$ AOMDL challenges $(X_0, \hat{X}_1, \dots, \hat{X}_t, X_1, X'_1, \dots, X_{q_s}, X'_{q_s})$. $\mathcal{B}$ has access to a discrete-logarithm oracle $\mathcal{O}^{\text{dl}}$, which it may query up to $2q_s + t$ times. $\mathcal{B}$ aims to output $(x_0, \hat{x}_1, \dots, \hat{x}_t, x_1, x'_1, \dots, x_{q_s}, x'_{q_s})$ such that $X_i = g^{x_i}$ for all i , with only $2q_s + t$ queries to its DL oracle.

To simulate $\mathcal{O}^{\text{dl}}$ queries by $\mathcal{C}$, $\mathcal{B}$ initializes a table Q_{dl} , which it uses to cache $\mathcal{O}^{\text{dl}}$ queries made by $\mathcal{C}$ and responses from $\mathcal{O}^{\text{dl}}$.

Execution. $\mathcal{B}$ then runs the general forking algorithm $\text{Fork}^{\mathcal{C}}(X)$ as described in Figure 6, on the simulator algorithm $\mathcal{C}$ and its instance X .

When $\mathcal{C}$ queries $\mathcal{O}^{\text{dl}}$, $\mathcal{B}$ queries its own oracle $\mathcal{O}^{\text{dl}}$. $\mathcal{B}$ caches the request and response in Q_{dl} , and then returns the response to $\mathcal{C}$.

With non-negligible probability lower-bounded by Equation 20, $\text{Fork}^C(X)$ will output the accepting answer $(I, \text{out}, \text{out}')$, such that:

$$\begin{aligned} h_I &= c^*, h'_I = c^{**}, \\ (\sigma^*, \text{aux}^*) &:= \text{out}, (\sigma^{**}, \text{aux}^{**}) := \text{out}' \\ (R^*, z^*) &:= \sigma^*, (R^*, z^{**}) := \sigma^{**}. \end{aligned}$$

Extracting the Discrete Logarithm of X_0 . We show that $\mathcal{B}$ can extract the discrete logarithm of X_0 from $\mathcal{A}$'s two valid forgeries. We assume without loss of generality that $\mathcal{A}$ queries H_{sig} on (PK, R^*, m^*) on its forgery in execution.

With overwhelming probability, $c^* \neq c^{**}$, and $\mathcal{B}$ can solve for $x_0 = \frac{z^* - z^{**}}{c^* - c^{**}}$. If $\mathcal{B}$ extracts x_0 , then we use this to extract a full AOMDL solution as follows.

Extracting an AOMDL Solution. $\mathcal{B}$ must now extract the remaining $(\hat{x}_1, \dots, \hat{x}_t, x_1, x'_1, \dots, x_{q_s}, x'_{q_s})$. Assume without loss of generality that $\mathcal{A}$ makes $t/2$ corruptions in each iteration. Recall that $\text{PK}_i = X_0 \hat{X}_1^i \dots \hat{X}_t^i$. Since t $\mathcal{O}^{\text{dl}}$ queries have been made on PK_i to obtain sk_i , together with x_0 , this gives $t+1$ points on the polynomial $f(Z) = x_0 + \hat{x}_1 Z + \dots + \hat{x}_t Z^t$ defining key generation. Thus, $\mathcal{B}$ can solve for $\hat{x}_1, \dots, \hat{x}_t$.

Now, for $x_1, x'_1, \dots, x_{q_s}, x'_{q_s}$ corresponding to each X_i , the method for extracting x_i will be one of the following cases, depending on how $\mathcal{A}$ queried $\mathcal{O}^{\text{Sign}_1}$ and $\mathcal{O}^{\text{Sign}_2}$ over the two iterations.

Case A. (X_i, X'_i) belongs to an honest party at the end of the two iterations.

Case 1 ((X_i, X'_i) has not appeared in an $\mathcal{O}^{\text{Sign}_2}$ query over the two iterations). In this case, X_i and X'_i have not yet been queried by $\mathcal{B}$. Thus, $\mathcal{B}$ queries $\mathcal{O}^{\text{dl}}$ directly to obtain x_i and x'_i . Two queries are made for each i in this case.

Case 2 ((X_i, X'_i) has appeared in an $\mathcal{O}^{\text{Sign}_2}$ query in a single iteration). A single query has been made by $\mathcal{B}$ to $\mathcal{O}^{\text{dl}}$ containing (X_i, X'_i) . If it occurred in the first iteration, then $((R_{k_j}, S_{k_j}), (i, z_{k_j}, \eta_j, a_j, c_j \lambda_{k_j}), \text{query}_j) \in \bar{\mathbf{Q}}_{\text{used}}$, where $\bar{\mathbf{Q}}_{\text{used}}$ denotes $\mathbf{Q}_{\text{used}}$ for the first iteration. (For FROST1, a_j here refers to $a_{k,j}$.) is such that $z_{k_j} = r_{k_j} + a_j s_{k_j} + c_j \lambda_{k_j} \text{sk}_{k_j}$. (Note that $\mathcal{B}$ can compute $f(k_j) = \text{sk}_{k_j}$.) To obtain a second value, $\mathcal{B}$ queries X'_i to $\mathcal{O}^{\text{dl}}$ and thus learns x'_i . Then $\mathcal{B}$ sets $x_i = z_{k_j} - a_j x'_i - c_j \lambda_{k_j} \text{sk}_{k_j}$. The case where the query occurred in the second iteration is similar. In total, two queries are made for each i in this case.

Now consider when (X_i, X'_i) appears in an $\mathcal{O}^{\text{Sign}_2}$ query in both iterations. Let query_j be that query from the first iteration and $\text{query}_{j'}$ from the second. such that $(X_i, X'_i) = (R_{k_j}, S_{k_j}) = (R_{k_{j'}}, S_{k_{j'}})$. Let

$$\begin{aligned} ((R_{k_j}, S_{k_j}), (i, z_{k_j}, \eta_j, a_j, c_j \lambda_{k_j}), \text{query}_j) &\in \bar{\mathbf{Q}}_{\text{used}} \\ ((R_{k_{j'}}, S_{k_{j'}}), (i, z_{k_{j'}}, \eta_{j'}, a_{j'}, c_{j'} \lambda_{k_{j'}}), \text{query}_{j'}) &\in \mathbf{Q}_{\text{used}} \end{aligned}$$

be the associated table entries for query_j and $\text{query}_{j'}$. Recall that I is the index such that $c^* = h_I$.

Case 3 ($\eta_j = \eta_{j'} < I$). In this case, a single query has been made, to obtain z_{k_j} in $((R_{k_j}, S_{k_j}), (i, z_{k_j}, \eta_j, a_j, c_j \lambda_{k_j}), \text{query}_j) \in \bar{\mathbf{Q}}_{\text{used}}$, such that $z_{k_j} = r_{k_j} + a_j s_{k_j} + c_j \lambda_{k_j} \mathbf{sk}_{k_j}$. To obtain a second value, $\mathcal{B}$ queries X'_i to $\mathcal{O}^{\text{dl}}$ and thus learns x'_i . Then $\mathcal{B}$ sets $x_i = z_{k_j} - a_j x'_i - c_j \lambda_{k_j} \mathbf{sk}_{k_j}$. In total, two queries are made for each i in this case.

Case 4 ($\eta_j \neq \eta_{j'}$ or $\eta_j = \eta_{j'} > I$). In this case, $a_j \neq a_{j'}$ and $\mathcal{B}$ has made two queries to obtain z_{k_j} and $z_{k_{j'}}$. $\mathcal{B}$ computes:

$$x'_i = \frac{z_{k'_{j'}} - z_{k_j} + c_j \gamma_{k_j} \mathbf{sk}_{k_j} - c_{j'} \gamma_{k_{j'}} \mathbf{sk}_{k'_{j'}}}{a_{j'} - a_j} \quad x_i = z_{k_j} - a_j x'_i - c_j \lambda_{k_j} \mathbf{sk}_{k_j} \quad (24)$$

Two queries are made for each i in this case.

Case B. (X_i, X'_i) belongs to a corrupt party at the end of the two iterations.

In this case, $\mathcal{B}$ has already made the queries in each of the cases above.

Thus, $\mathcal{B}$ has extracted x_i for all X_i using exactly $2q_s + t$ queries and returns $(x_0, \hat{x}_1, \dots, \hat{x}_t, x_1, x'_1, \dots, x_{q_s}, x'_{q_s})$ and win the $2q_s + t + 1$ -aomdl game. Combining Equations 20 and 23 gives Equation 2.

B Proof of Lemma 2

Let $A_2(n, d, w)$ denote the maximum size of a code $\mathcal{C} \subseteq \{0, 1\}^n$ where all codewords have weight w , and each pair has distance at least d . Clearly $A_2(n, d, w) = 1$ if $d > 2w$, since two codewords of weight w have distance at most $2w$.

Assume $d \leq 2w$ and pick $\mathcal{C}$ such that $|\mathcal{C}| = A_2(n, d, w)$. For $i \in [n]$, we let $\mathcal{C}_i \in \{0, 1\}^{n-1}$ be the code obtained by taking all $\mathbf{c} \in \mathcal{C}$ such that $\mathbf{c}[i] = 1$, and then removing the i th coordinate. Note that the codewords in $\mathcal{C}_i$ all have weight $w - 1$, and the distance remains at least d . Let $M_i = |\mathcal{C}_i|$. Note that

$$\sum_{i=1}^n M_i = w \cdot |\mathcal{C}| = w \cdot A_2(n, d, w) ,$$

since every codeword in $\mathcal{C}$ is added to exactly w sets $\mathcal{C}_i$. On the other hand, $M_i \leq A_2(n - 1, d, w - 1)$, and therefore

$$A_2(n, d, w) \leq \frac{n}{w} A_2(n - 1, d, w - 1) .$$

For the specific case where $d = 2e$, and $w \geq e$, we can iterate this argument exactly $w - e + 1$ times to obtain

$$A_2(n, 2e, w) \leq \left(\prod_{i=0}^{w-e} \frac{n-i}{w-i} \right) A_2(n - (w - e) - 1, 2e, e - 1) = \left(\prod_{i=0}^{w-e} \frac{n-i}{w-i} \right) ,$$

since $A_2(n - (w - e) - 1, 2e, e - 1) = 1$.